

Microservicios: del monolito a servicios autónomos

Parte 1 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Qué es un microservicio, dónde se cortan sus fronteras y qué cuesta vivir con ellos

El monolito de la Corporación Industrial del Caribe

Todo vive en un solo paquete: lo bueno es lo simple, lo malo es que todo depende de todo.



1 Un despliegue al mes

Cada cambio obliga a probar y publicar todo el sistema.

2 El fallo se contagia

Si Reportes agota la memoria, también cae Pedidos.

3 Escalar es copiar todo

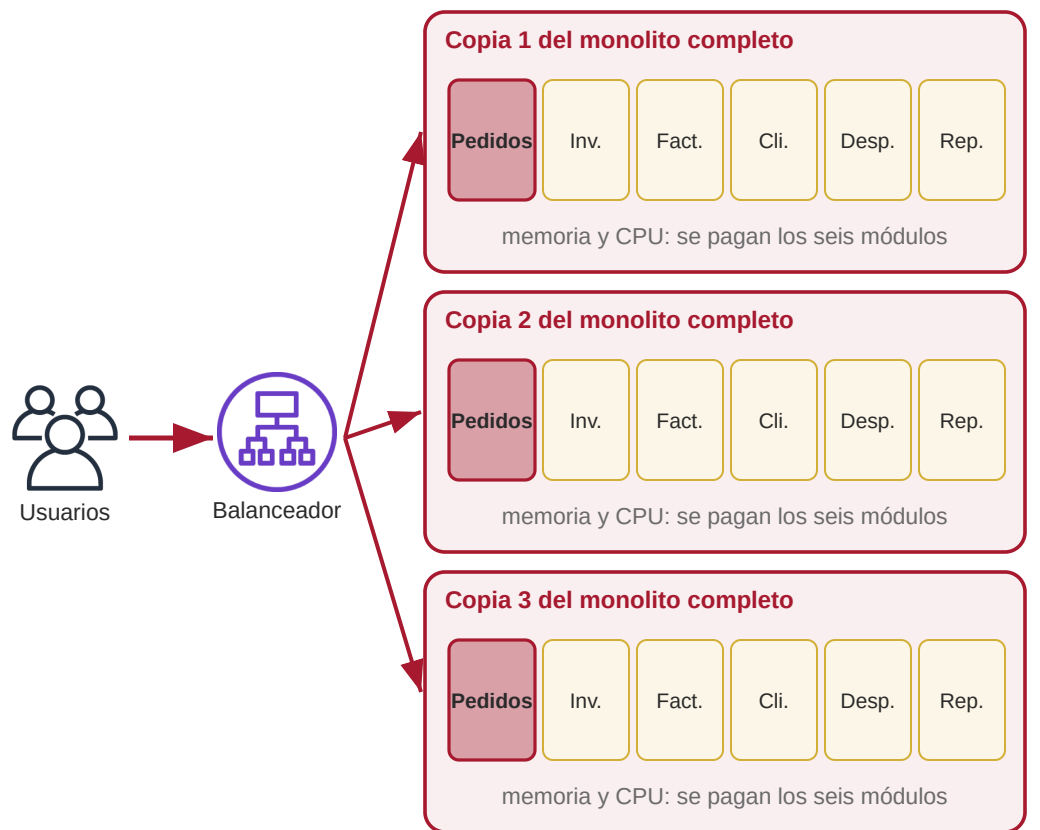
Aunque solo Pedidos esté saturado, se duplica el paquete completo.

4 Una sola tecnología

Todos los equipos comparten lenguaje, versión y base de datos.

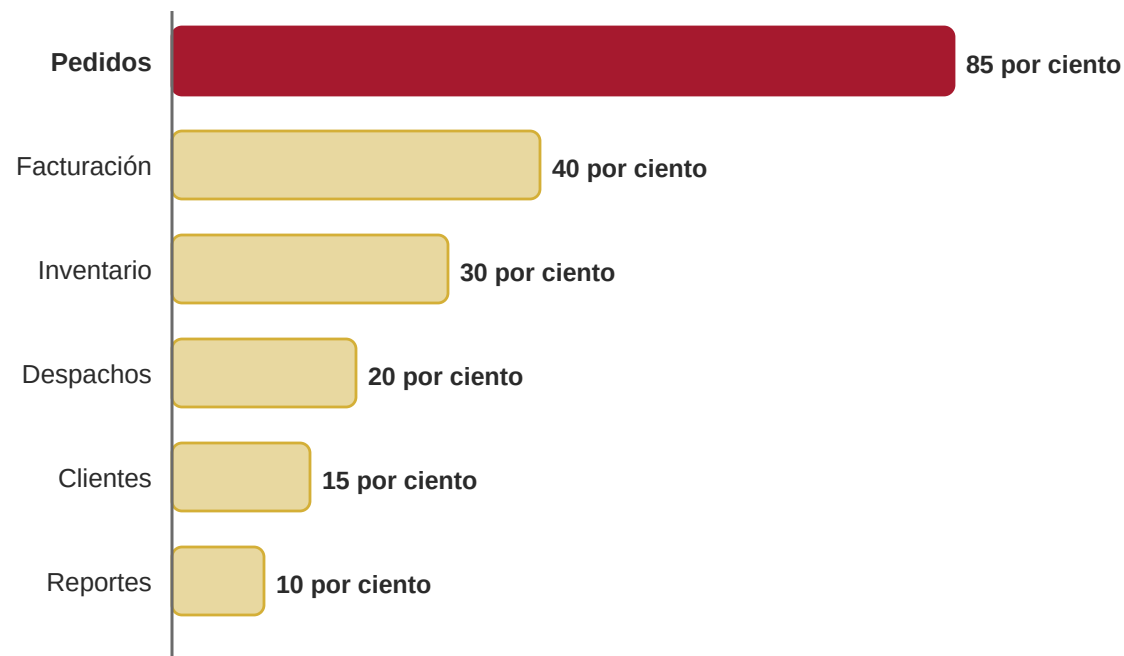
Escalar un monolito: se copia todo, aunque falte uno solo

Pedidos se satura en temporada alta, pero la única palanca es duplicar la aplicación completa.



Tres copias completas para sostener un solo módulo

Uso de CPU por módulo en temporada alta (ilustrativo)



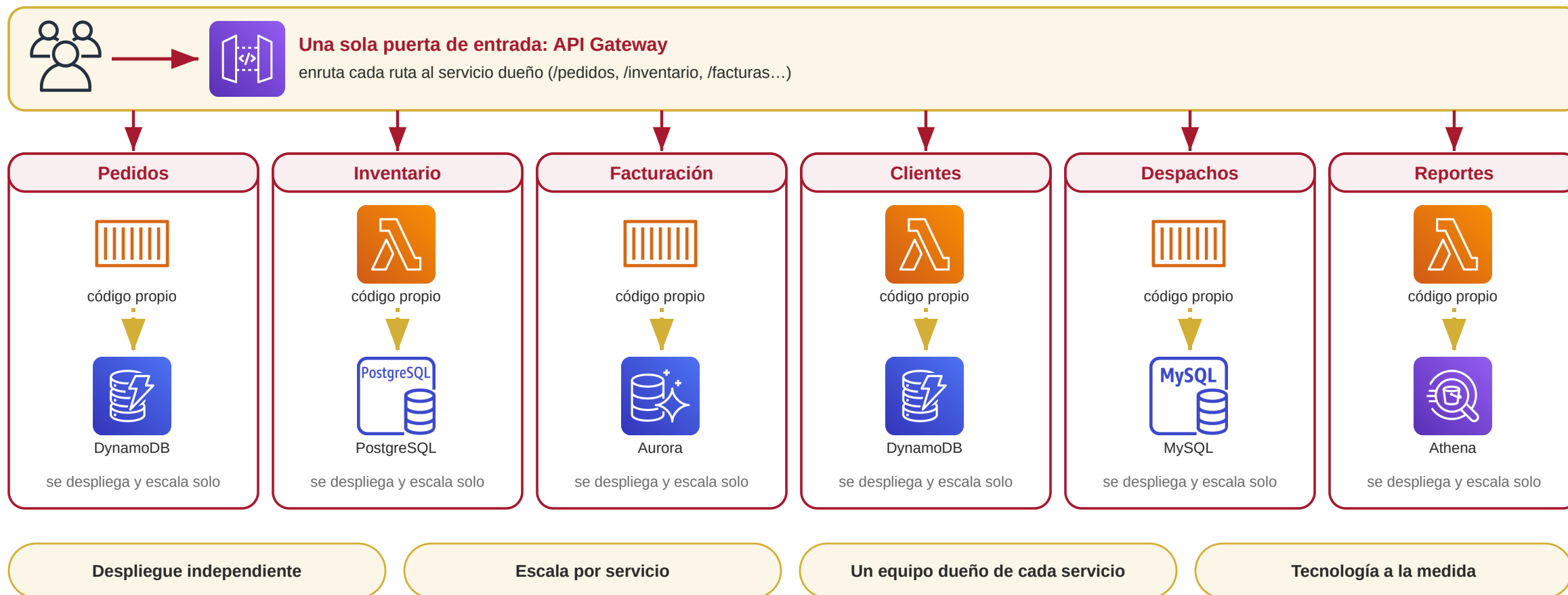
Escalar horizontalmente un monolito es todo o nada:

solo Pedidos necesita capacidad, pero hay que replicar también Reportes, Clientes y los demás módulos que están casi ociosos.

Se desperdicia dinero y se escala a ciegas.

Microservicios: servicios pequeños, autónomos y acoplados con holgura

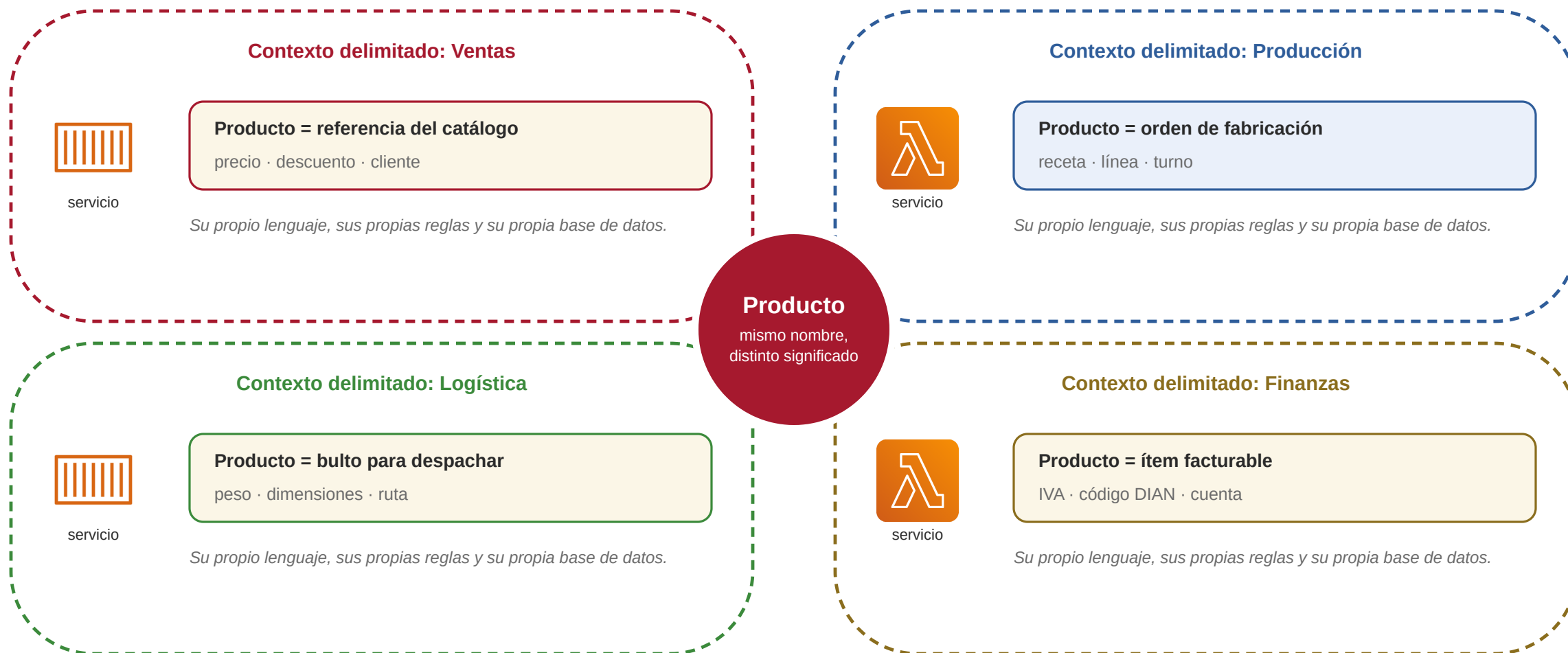
Cada servicio hace una cosa del negocio, tiene sus propios datos y se despliega sin pedir permiso a los demás.



Pequeño no significa fragmento al azar: cada servicio encierra una capacidad completa del negocio.

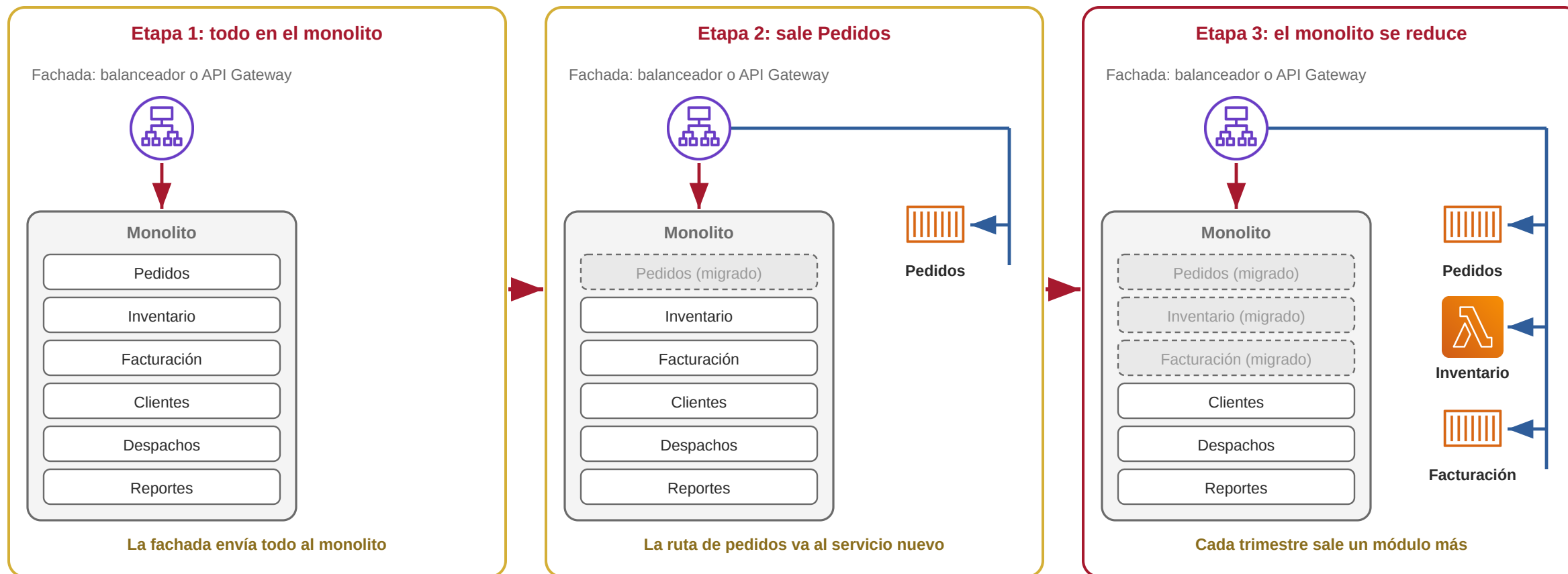
Límites de dominio: cada servicio es dueño de un contexto

La palabra Producto significa cosas distintas en Ventas, Producción, Logística y Finanzas; cada contexto tiene su modelo.



Cómo partir el monolito: el patrón estrangulador

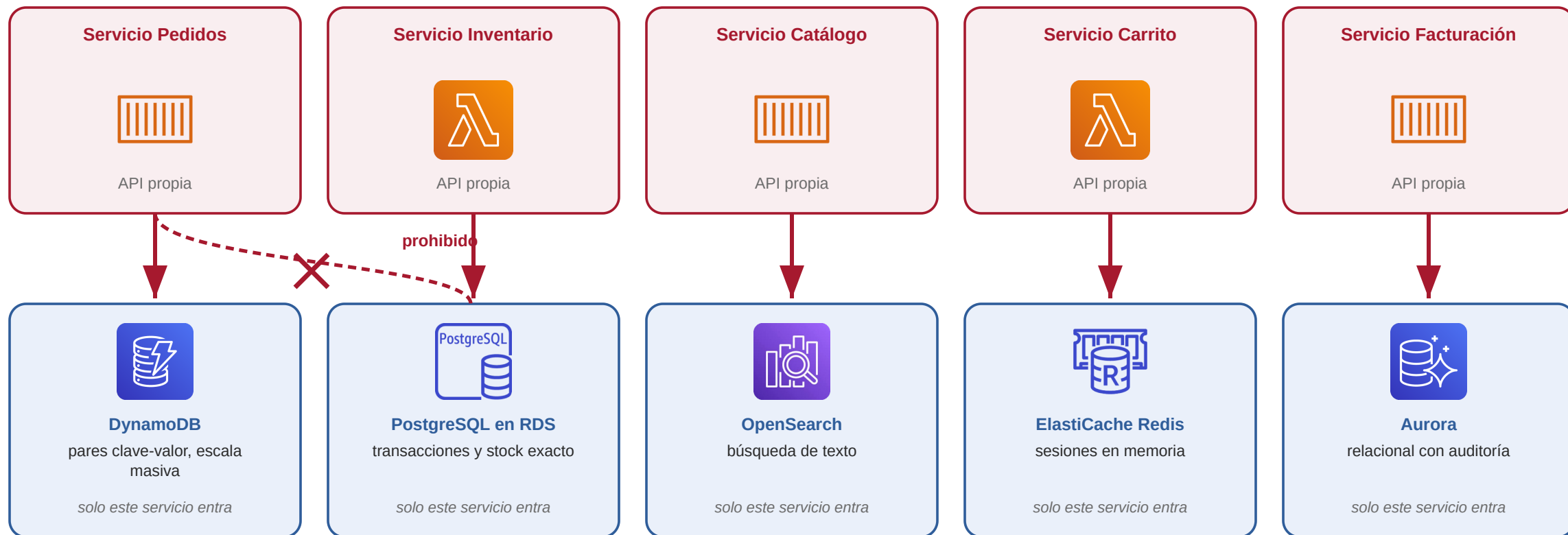
Se migra un módulo a la vez; la fachada decide a quién llega cada petición.



Patrón estrangulador: la aplicación nueva crece alrededor del monolito, como la higuera que envuelve al árbol, hasta que el monolito ya no hace falta. Sin reescribir todo de golpe y sin parar el negocio.

Una base de datos por servicio

Compartir tablas es el acoplamiento más peligroso; cada servicio guarda lo suyo y expone una API.

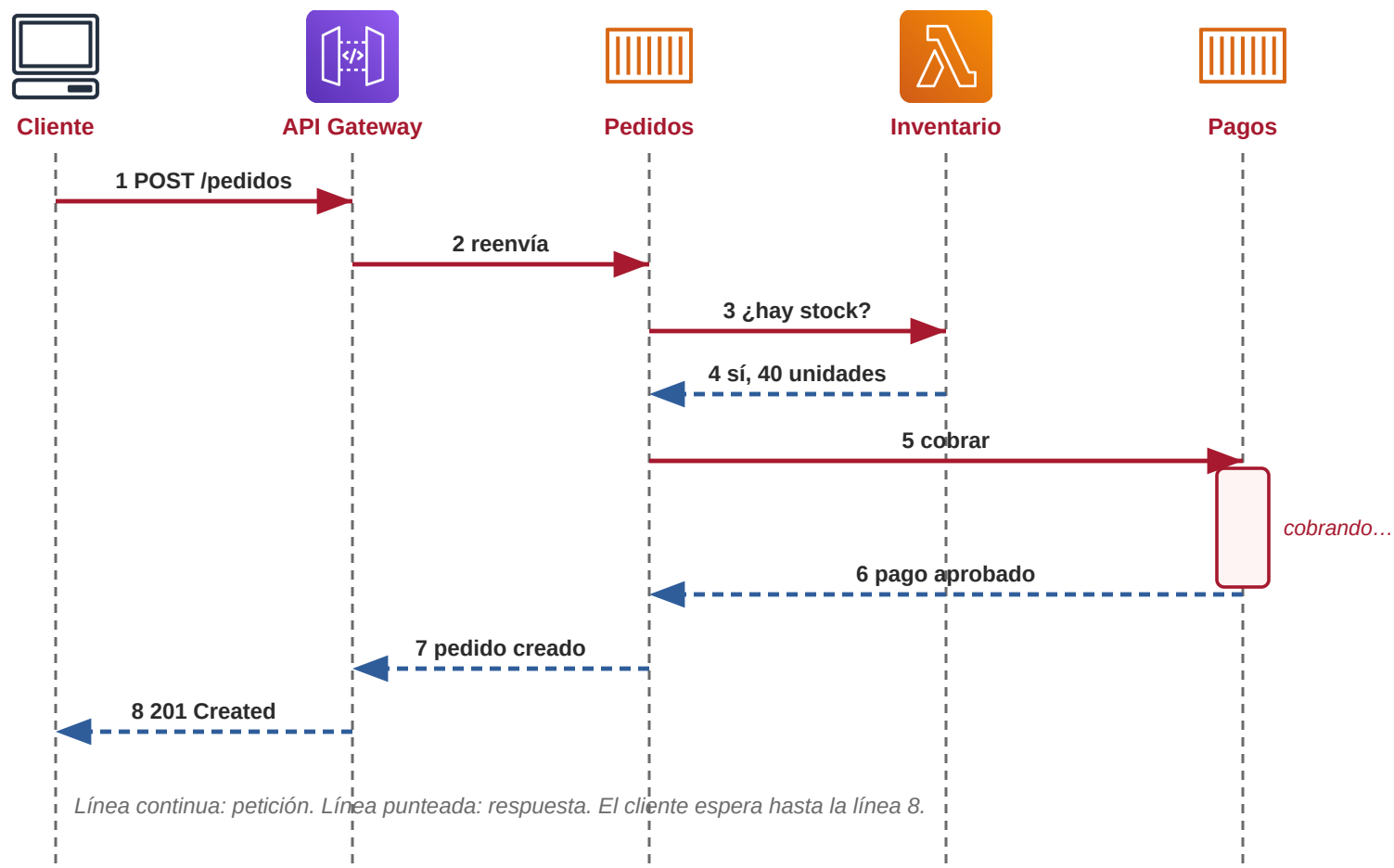


Persistencia polígloa: cada servicio usa el motor que mejor encaja con sus datos.

Si Pedidos necesita saber el stock, lo pide por la API de Inventario o escucha un evento; nunca consulta las tablas de Inventario directamente, porque eso volvería a atar los dos servicios.

Comunicación síncrona: petición y respuesta

Simple de entender, pero el cliente espera a toda la cadena y la disponibilidad se multiplica.



La latencia se suma

30 ms	45 ms	80 ms
Pedidos	Inventario	Pagos
$30 + 45 + 80 = 155 \text{ ms}$ (ilustrativo)		

REST sobre HTTP: simple y universal

gRPC: binario, rápido, entre servicios

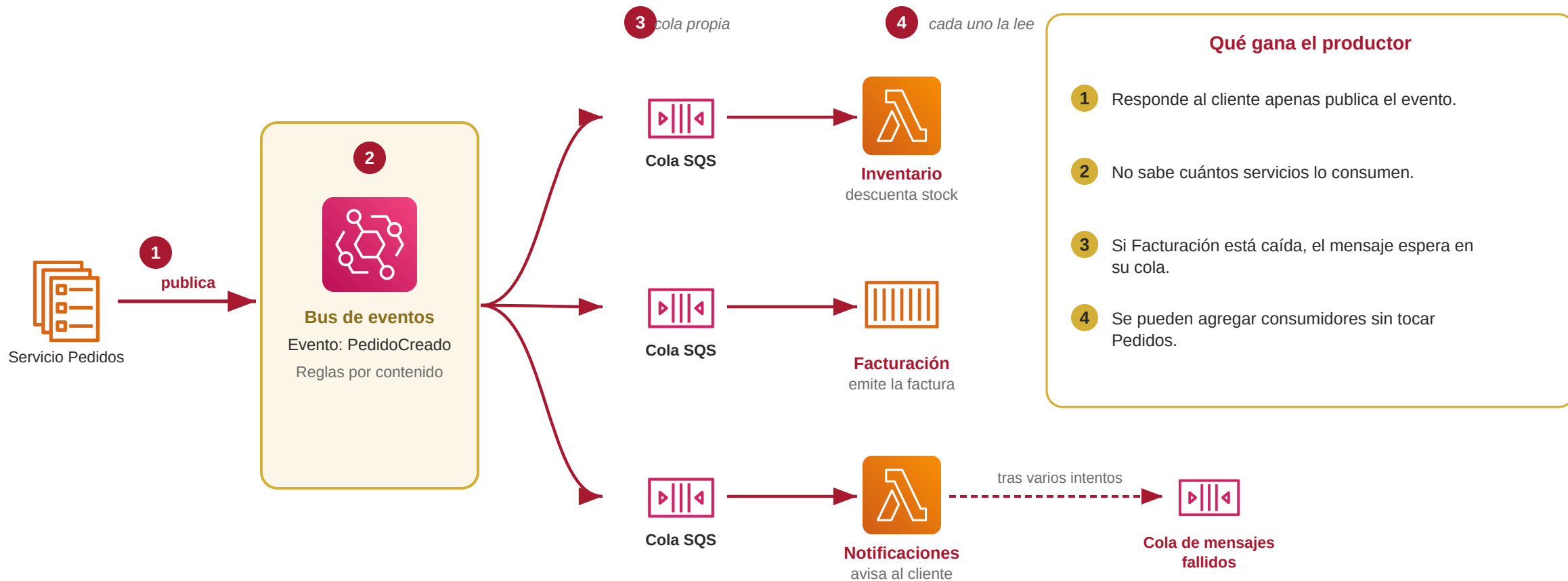
Riesgos de la cadena síncrona

- Si Pagos tarda 5 segundos, el cliente espera 5 segundos.
- Si Pagos cae, Pedidos también falla.
- Todos deben estar disponibles a la vez.

Úsala cuando el cliente necesita la respuesta ya

Comunicación asíncrona: eventos y colas

El servicio publica un hecho y sigue con su vida; cada interesado lo procesa a su ritmo.

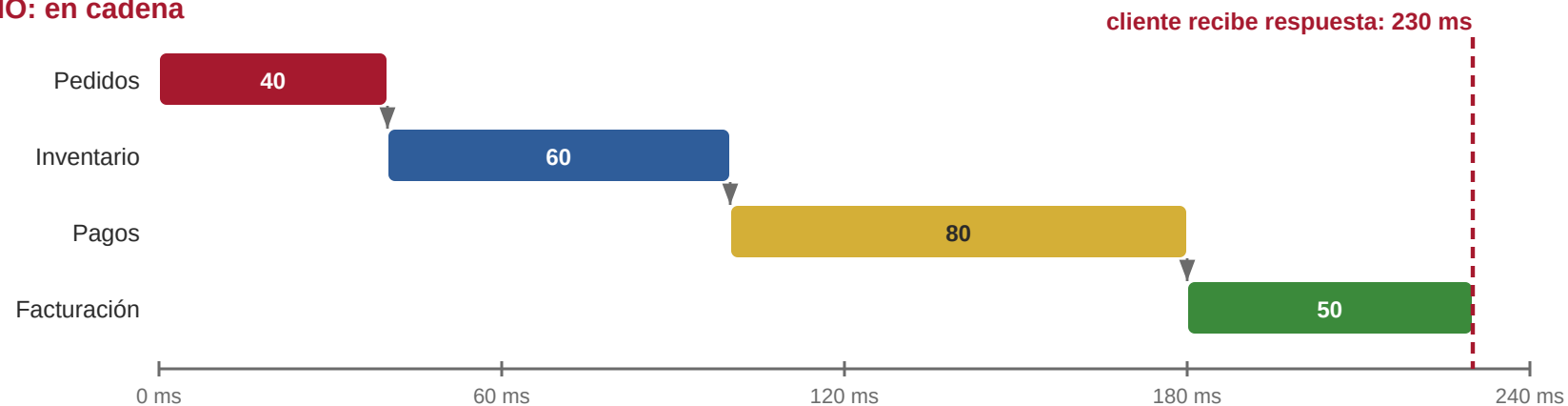


Acoplamiento temporal roto: productor y consumidores no tienen que estar activos al mismo tiempo (patrón visto en la Semana 13).

Síncrono frente a asíncrono: el mismo pedido en el tiempo

El cliente percibe 230 ms si todo va en cadena y 40 ms si el resto se procesa en paralelo.

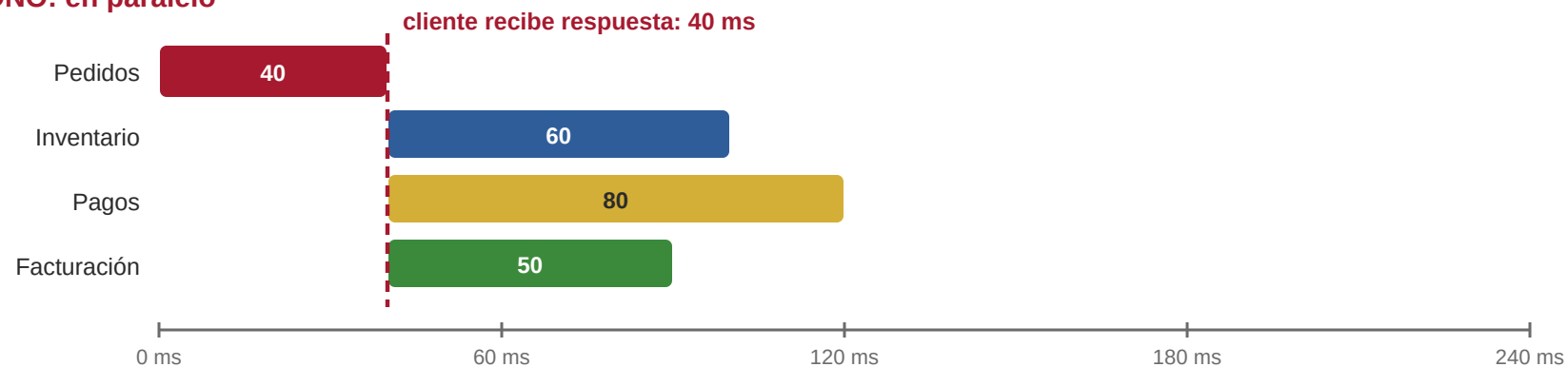
SÍNCRONO: en cadena



Elige síncrono si...

- el usuario necesita el dato ya
- es una consulta, no un cambio
- la cadena es corta

ASÍNCRONO: en paralelo



Elige asíncrono si...

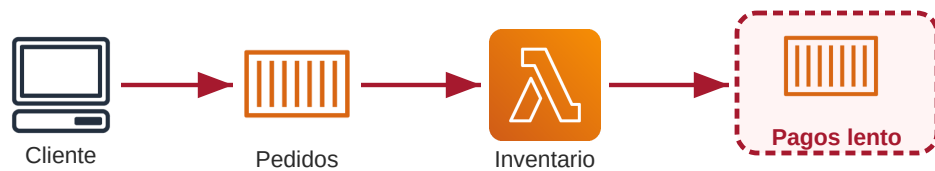
- el trabajo puede esperar
- varios servicios reaccionan al mismo hecho
- hay picos y conviene amortiguar

Tiempos ilustrativos para comparar los dos modelos.

Fallos en cascada y cómo contenerlos

En microservicios la red falla: se diseña para que un servicio enfermo no contagie a los demás.

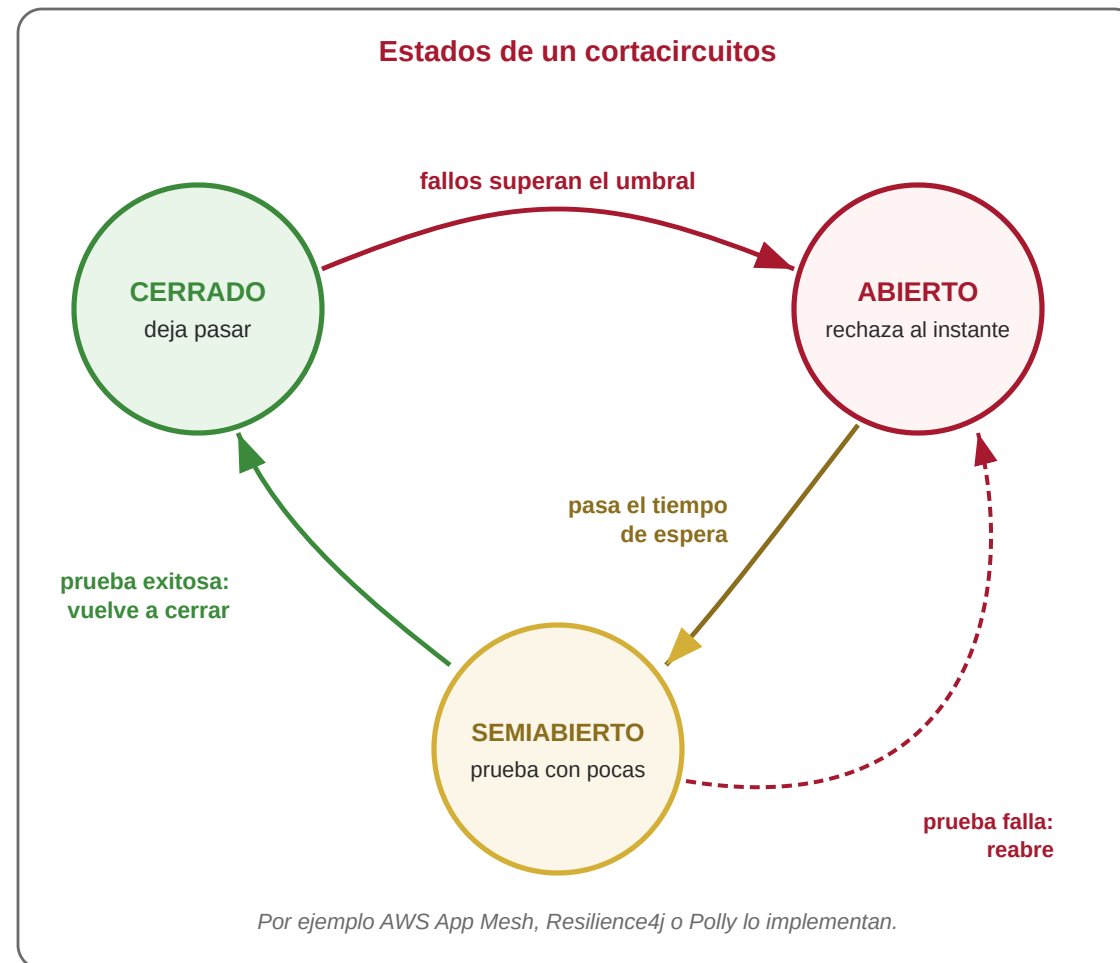
Un servicio lento arrastra a toda la cadena



Cada llamada espera... y mientras espera retiene un hilo y memoria.

Los hilos se agotan, Pedidos también deja de responder y la falla sube hasta el cliente.

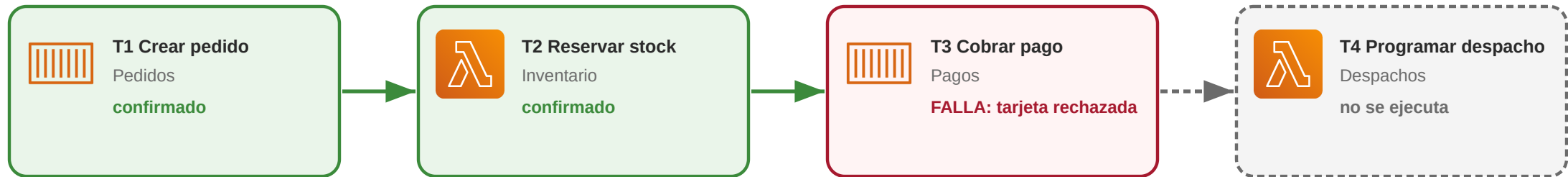
- 1 **Tiempo límite**
Si no responde en 2 segundos, se corta; nunca esperar sin límite.
- 2 **Reintento con retroceso**
Reintenta esperando 1, 2, 4 segundos más un azar (jitter) para no avalanchar.
- 3 **Cortacircuitos**
Tras varios fallos deja de llamar y responde rápido con un error o un valor por defecto.
- 4 **Mamparos (bulkhead)**
Grupos de hilos separados por dependencia: un servicio lento no ahoga a los demás.



Consistencia sin transacciones globales: el patrón Saga

Sin una transacción única entre bases, cada paso confirma y, si algo falla, otro paso lo deshace.

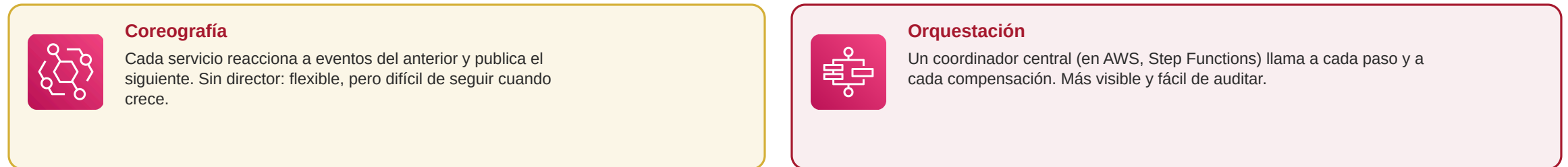
Camino feliz: una transacción local por servicio



Compensaciones: cada paso sabe deshacerse

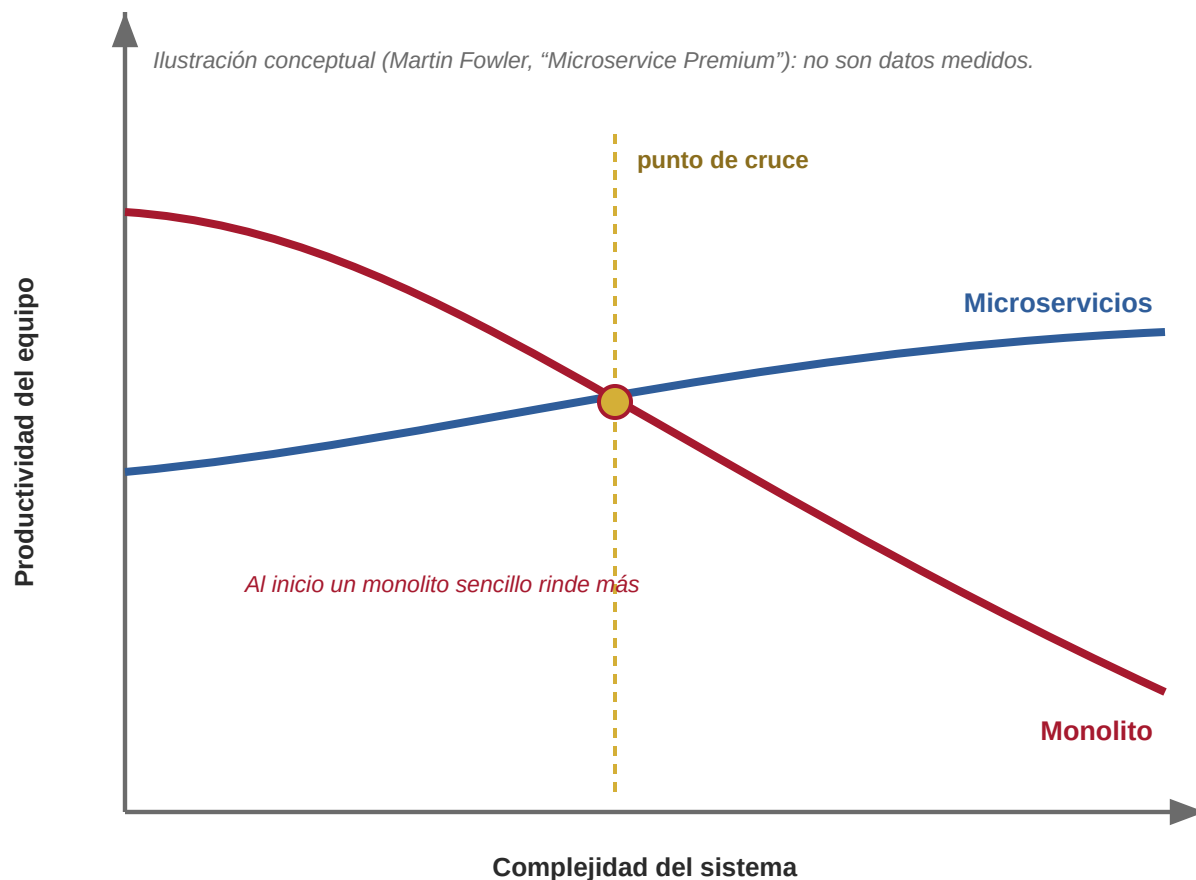


Dos formas de coordinar la saga



Ventajas y costos: la prima de los microservicios

Dividir ayuda cuando el sistema ya es complejo; antes de eso, solo agrega peso.



Lo que ganas

Despliegue independiente

Escala por servicio

Fallos aislados

Equipos autónomos

Tecnología a la medida

Cada servicio es un sistema distribuido que alguien debe operar de noche.

Lo que pagas

Llamadas por red y latencia

Observabilidad distribuida

Consistencia eventual

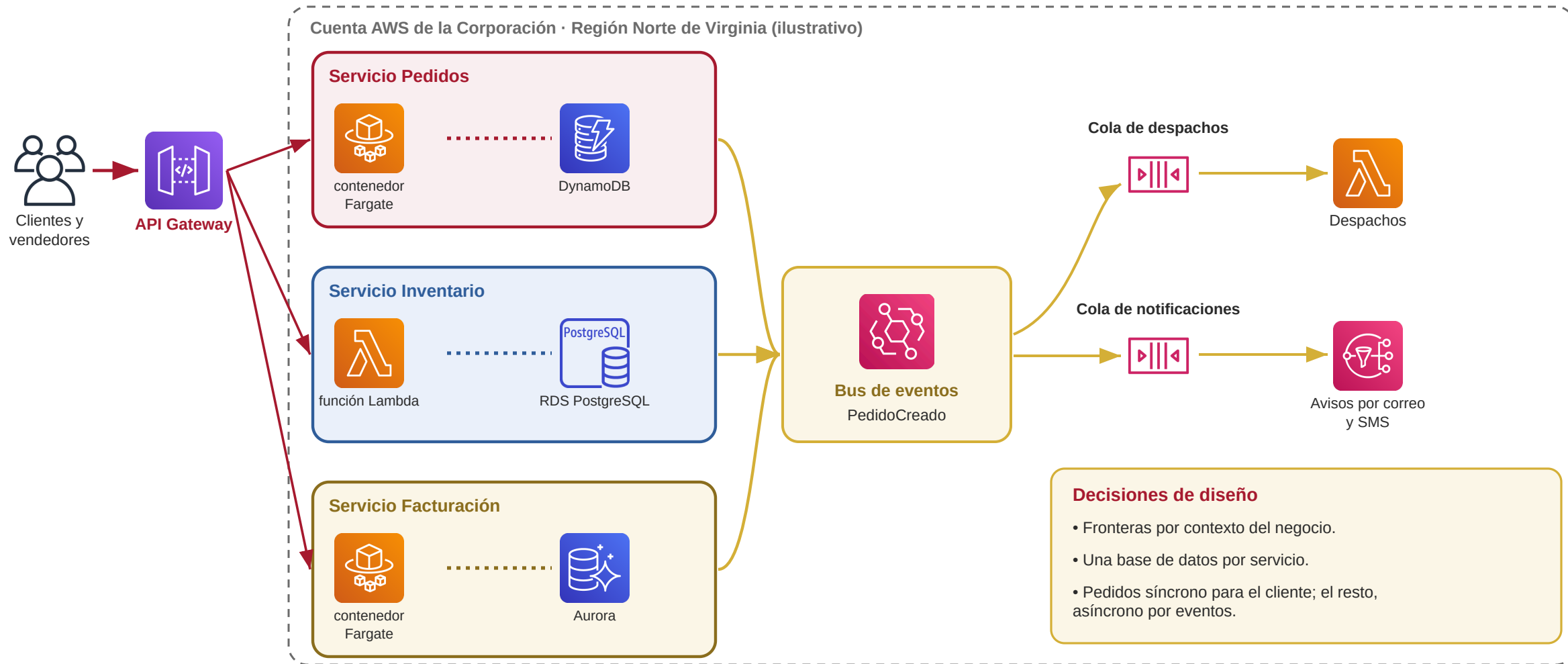
Muchos despliegues que operar

Pruebas entre servicios

Cuando el sistema es complejo, la ganancia supera al costo; cuando es pequeño, no.

Caso: la arquitectura objetivo de la Corporación Industrial del Caribe

Tres servicios propios, un bus de eventos y dos consumidores asíncronos detrás de una sola API.



Lo que queda de la Parte 1

Microservicios son una decisión de negocio y de organización antes que de tecnología.

1



El monolito no es el villano

Simple al inicio; duele al crecer.

2



Cortar por el dominio

Contextos delimitados, no capas técnicas.

3



Datos propios

Una base de datos por servicio;
sin JOIN ajenos.

4



Hablar con cuidado

Síncrono si se espera; eventos si
puede esperar.

Siguiente pregunta: ¿cómo se empaqueta y se ejecuta cada servicio?



Imagen



Registro



Contenedor



Orquestador



Sin servidores

Parte 2: contenedores y orquestación.

Docker, ECR, Kubernetes, ECS, EKS y Fargate.

Contenedores: empaquetar, ejecutar y orquestar

Parte 2 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

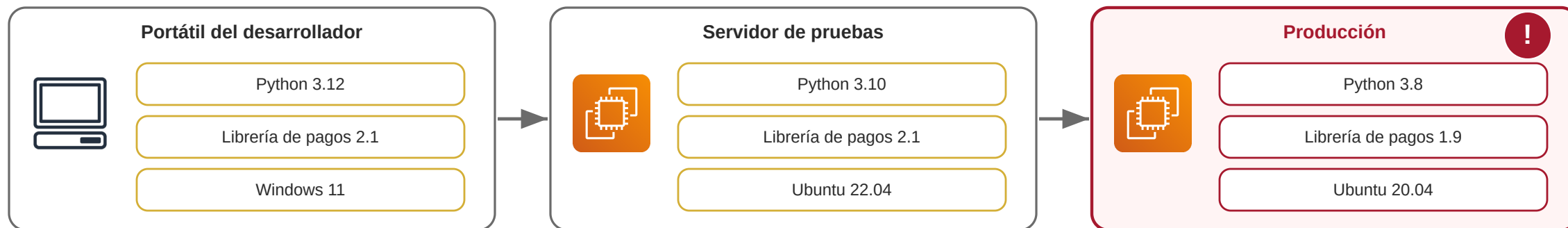
Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Docker, ECR, Kubernetes, ECS, EKS y Fargate: de la imagen al clúster

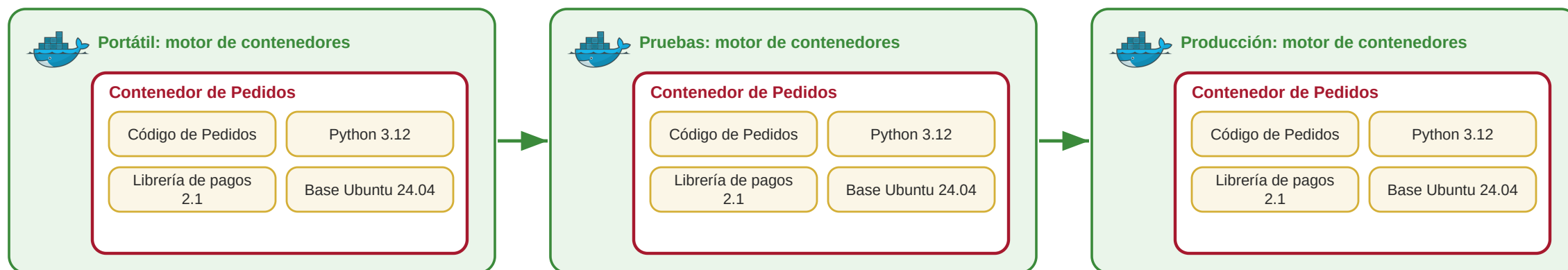
“En mi máquina sí funciona”: el problema que resuelven los contenedores

Un contenedor empaqua la aplicación con su entorno y elimina las diferencias entre portátil, pruebas y producción.

Sin contenedores: cada entorno se arma a mano y termina distinto



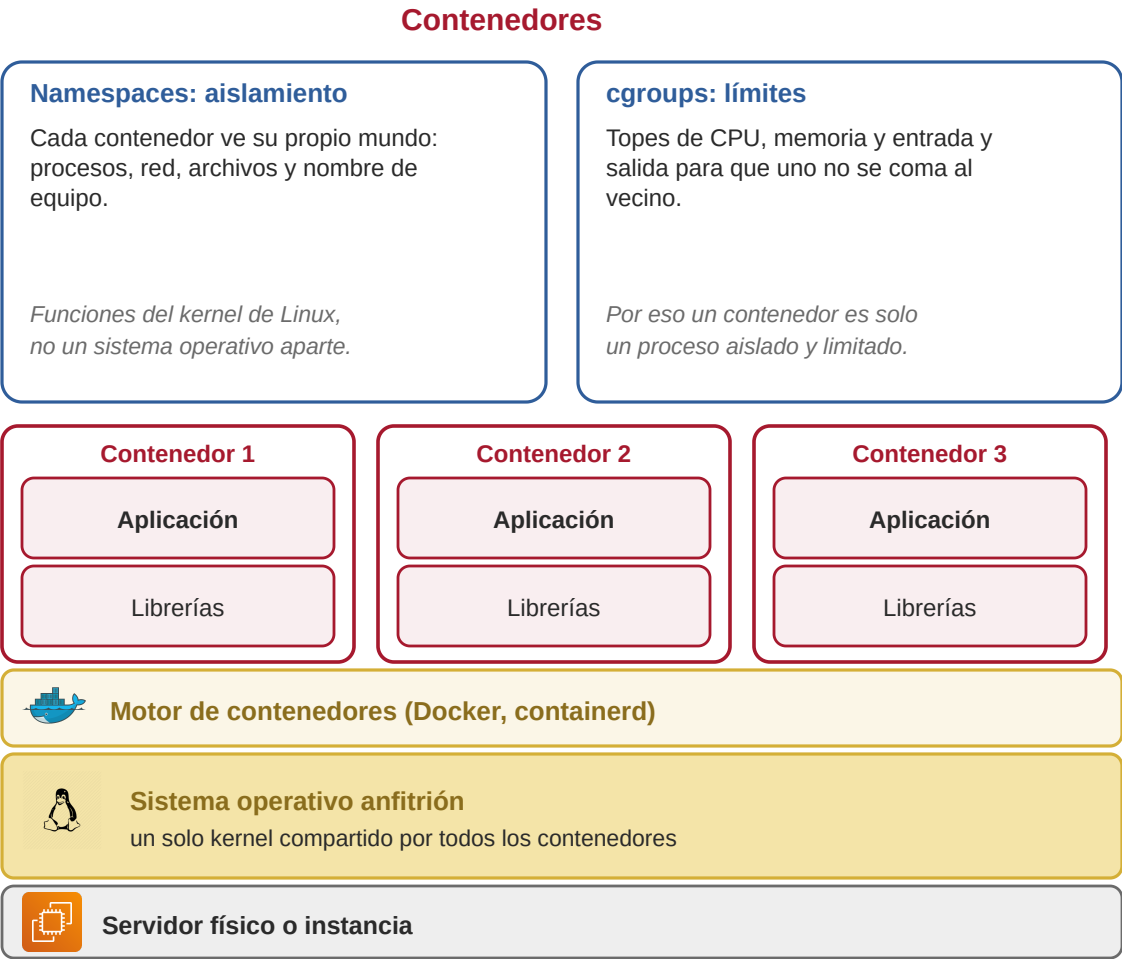
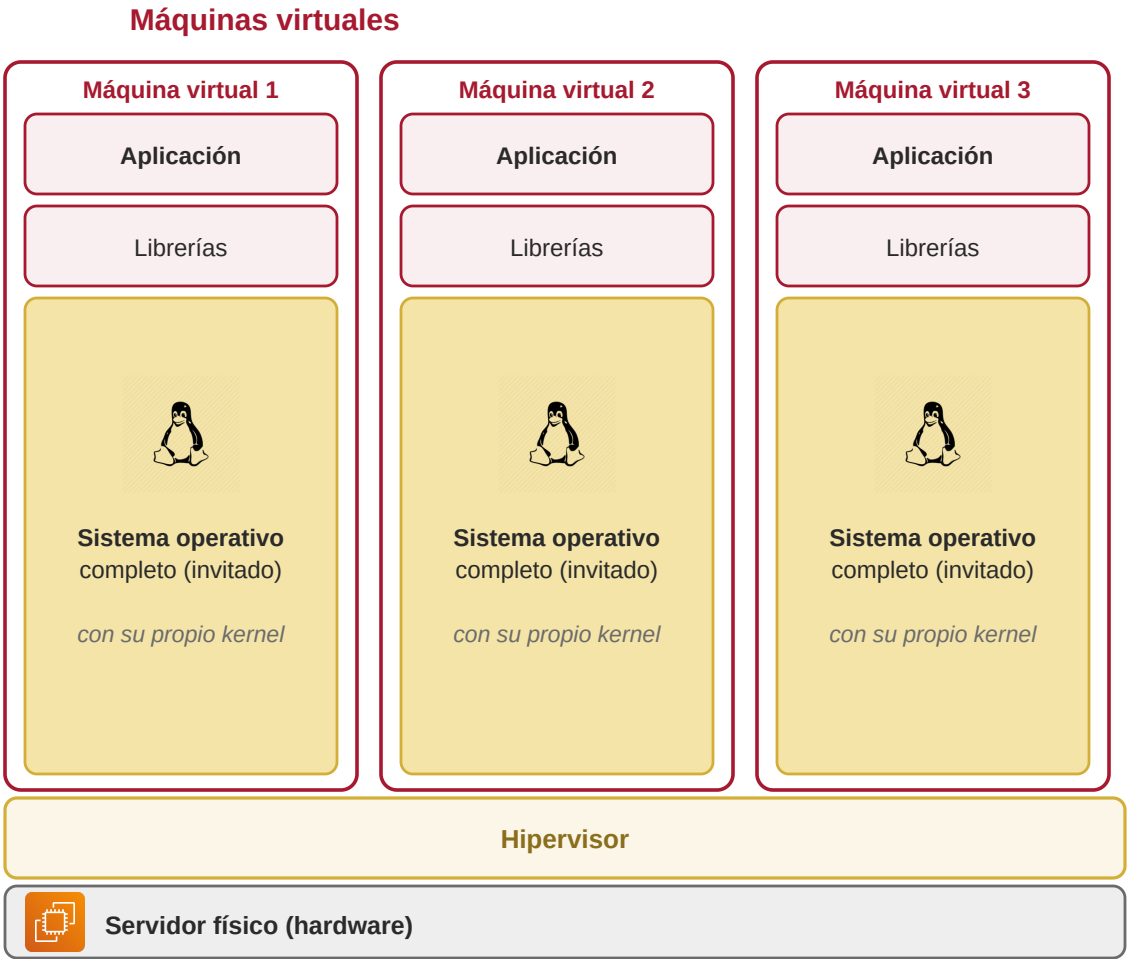
Con contenedores: la aplicación viaja con su entorno



La imagen se construye una vez y se ejecuta idéntica en cualquier lugar donde haya un motor de contenedores.

Contenedor frente a máquina virtual

La VM virtualiza el hardware y trae un sistema operativo completo; el contenedor comparte el kernel y pesa mucho menos.



La imagen: capas de solo lectura y una capa de escritura

Cada instrucción del Dockerfile agrega una capa; los contenedores comparten las capas y solo escriben en la suya.

Dockerfile

- 1 FROM python:3.12-slim
- 2 WORKDIR /app
- 3 COPY requirements.txt .
- 4 RUN pip install -r requirements.txt
- 5 COPY . .
- 6 CMD ["python", "app.py"]

Cada instrucción de archivos crea una capa.

docker build

Contenedor en ejecución

Capa de escritura del contenedor

efímera: se pierde al borrarlo

4 COPY . .

código de Pedidos

3 RUN pip install

dependencias de la aplicación

2 COPY requirements.txt

lista de dependencias

1 FROM python:3.12-slim

capa base: sistema mínimo + Python

Imagen:

capas de solo lectura, apiladas

CMD: instrucción de arranque (metadato)

Las capas se comparten

Tres contenedores usan la misma imagen:

Imagen pedidos:1.0 (solo lectura)

contenedor
1

contenedor
2

contenedor
3

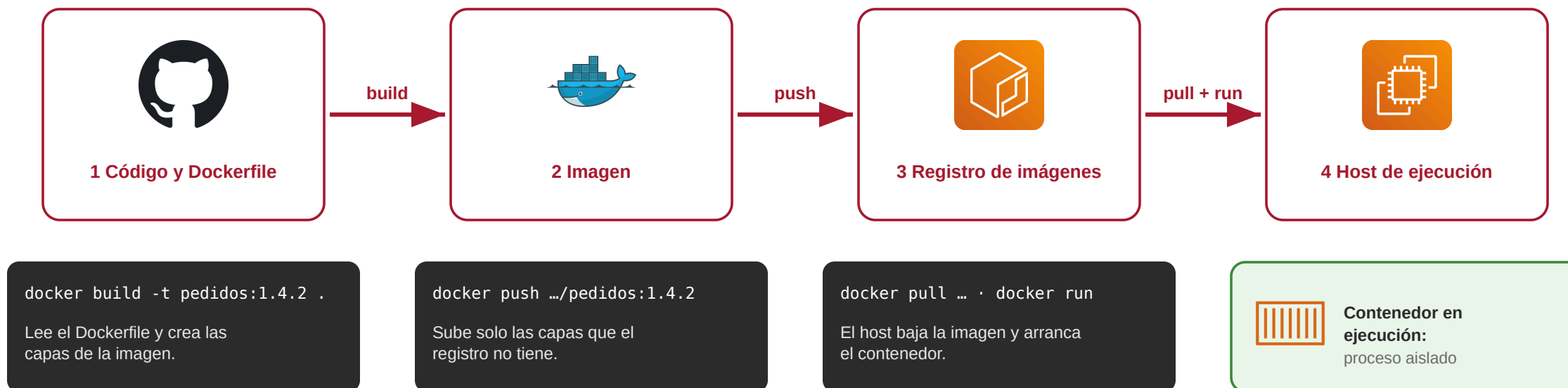
Caché de construcción

Si solo cambias app.py, Docker reutiliza las capas 1 a 3 y reconstruye solo la capa 4: la compilación tarda segundos.

Ordena el Dockerfile de lo que menos cambia a lo que más cambia.

De Dockerfile a contenedor: construir, subir, bajar y ejecutar

Cuatro verbos: build, push, pull y run; la imagen es el artefacto que viaja entre ellos.



Tres nombres que conviene distinguir

Etiqueta (tag)

pedidos:1.4.2

Nombre legible de una versión. Se puede mover a otra imagen.

Digest

sha256:9f2c...

Huella única del contenido; nunca cambia. Úsala para fijar versiones.

latest

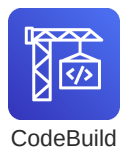
pedidos:latest

Etiqueta por defecto. Evítala en producción: no sabes qué versión es.

Registro de imágenes: Amazon ECR

El registro guarda, versiona, escanea y entrega las imágenes a quien tenga permiso.

Publican (push)



Amazon ECR

registro privado y administrado

Repositorio cic/pedidos

1.4.0

1.4.1

1.4.2

latest

Cada etiqueta apunta a una imagen con su digest.
Inmutabilidad de etiquetas: impide sobrescribir 1.4.2.
Los repositorios son privados por defecto.



Escaneo de vulnerabilidades



Cifrado con KMS



Permisos con IAM



Ciclo de vida y réplica

Descargan (pull)



ECS



EKS



Lambda (imagen)



Fargate

Otros registros: el concepto es el mismo en todas las nubes



Docker Hub
público y privado



Azure Container Registry
en Azure



Google Artifact Registry
en Google Cloud



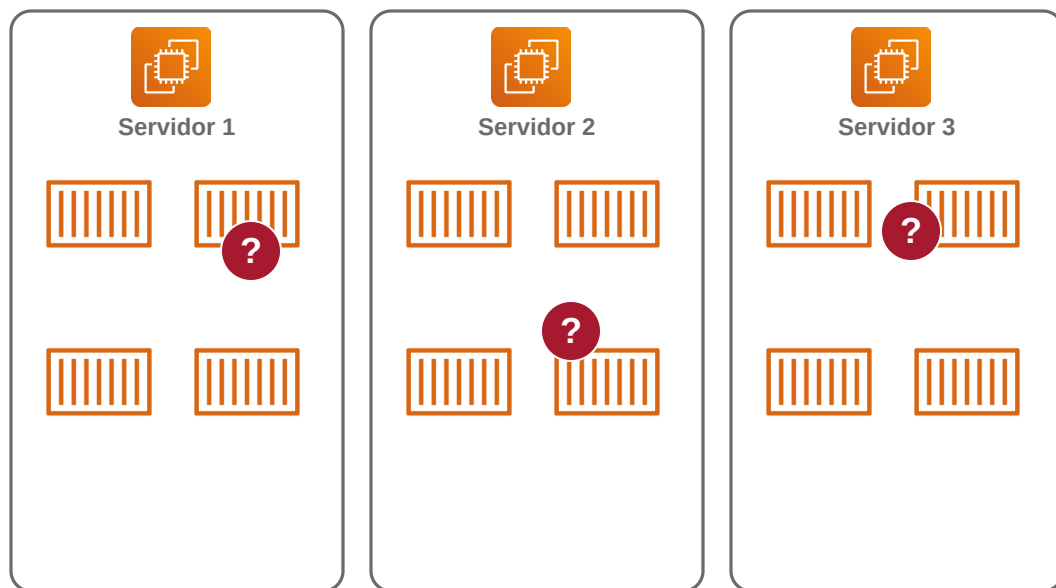
Amazon ECR
en AWS: nuestra práctica

Regla del curso: las imágenes de producción vienen de un registro propio y privado, nunca de una imagen cualquiera de Internet.

Un contenedor no basta: ¿por qué orquestar?

Cuando los contenedores son decenas o cientos, alguien tiene que programarlos, vigilarlos y actualizarlos.

Sin orquestador: 12 contenedores a mano



- ¿En qué servidor cabe el siguiente contenedor?
- ¿Quién lo reinicia si se cae a las 3 de la mañana?
- ¿Cómo se actualiza la versión sin cortar el servicio?
- ¿Cómo lo encuentran los demás servicios si cambia de IP?



Con orquestador

Kubernetes, Amazon ECS o Amazon EKS



Programar

Elige en qué nodo corre cada contenedor según CPU y memoria libre.



Autorreparar

Si un contenedor o nodo falla, crea otro automáticamente.



Escalar

Sube o baja réplicas según la carga medida.



Descubrir y balancear

Da un nombre estable y reparte el tráfico entre réplicas.



Actualizar sin cortes

Cambia versión por oleadas (rolling update) y puede revertir.

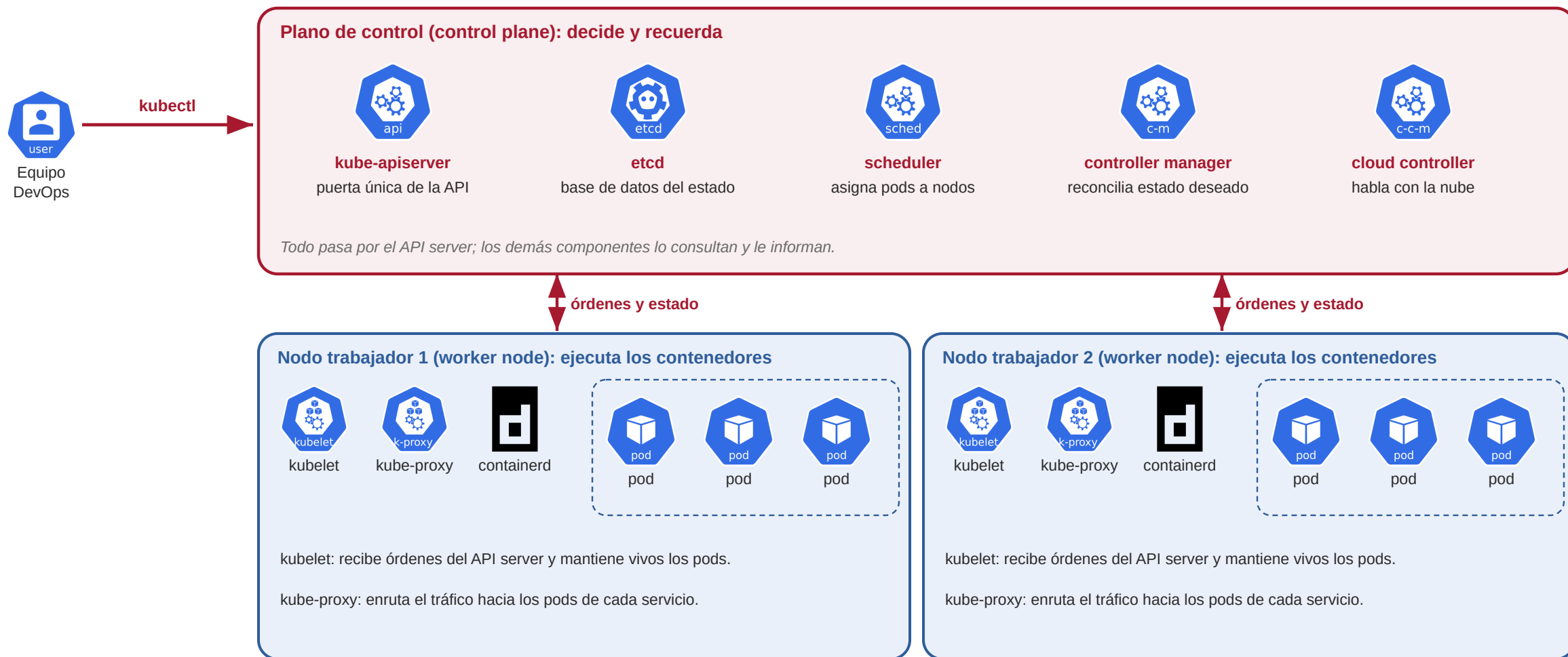


Configurar y proteger

Injecta variables y secretos sin meterlos en la imagen.

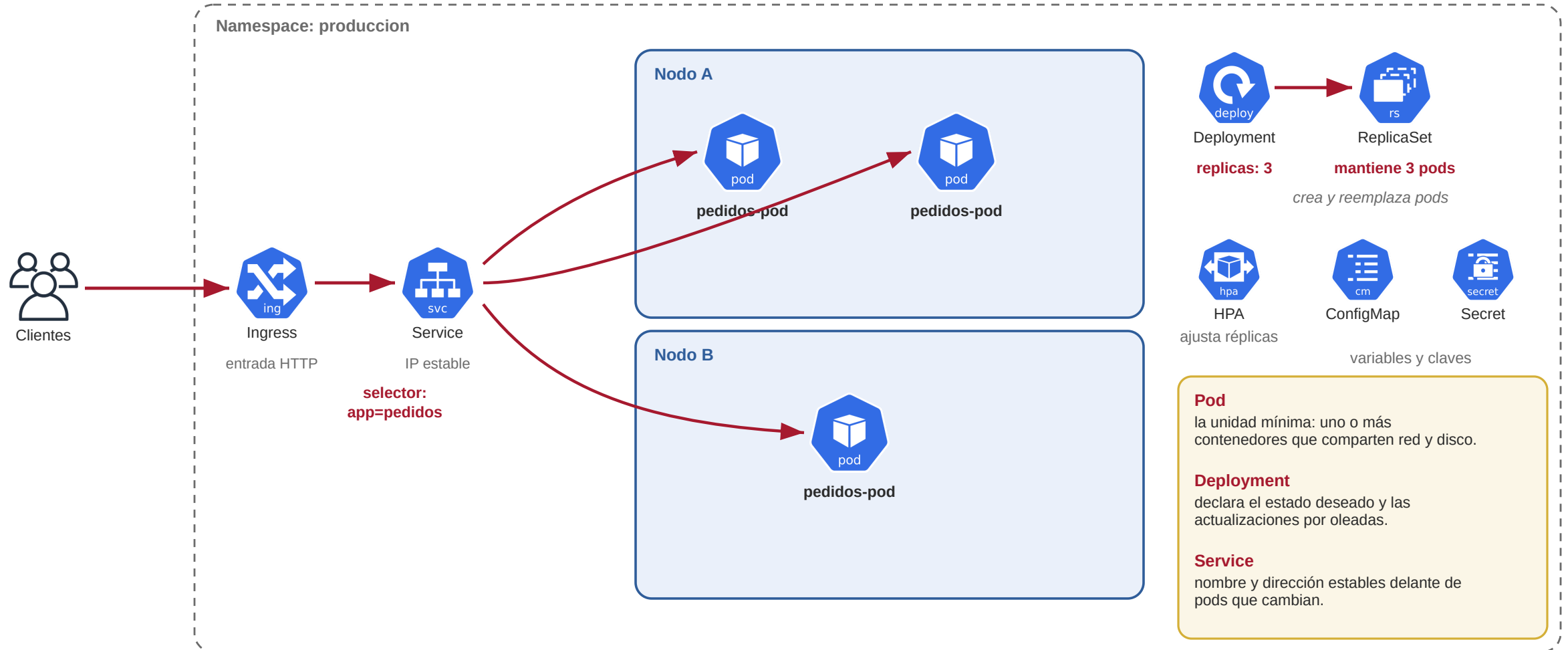
Kubernetes: plano de control y nodos trabajadores

Un cerebro que guarda el estado deseado y nodos que lo hacen realidad ejecutando pods.



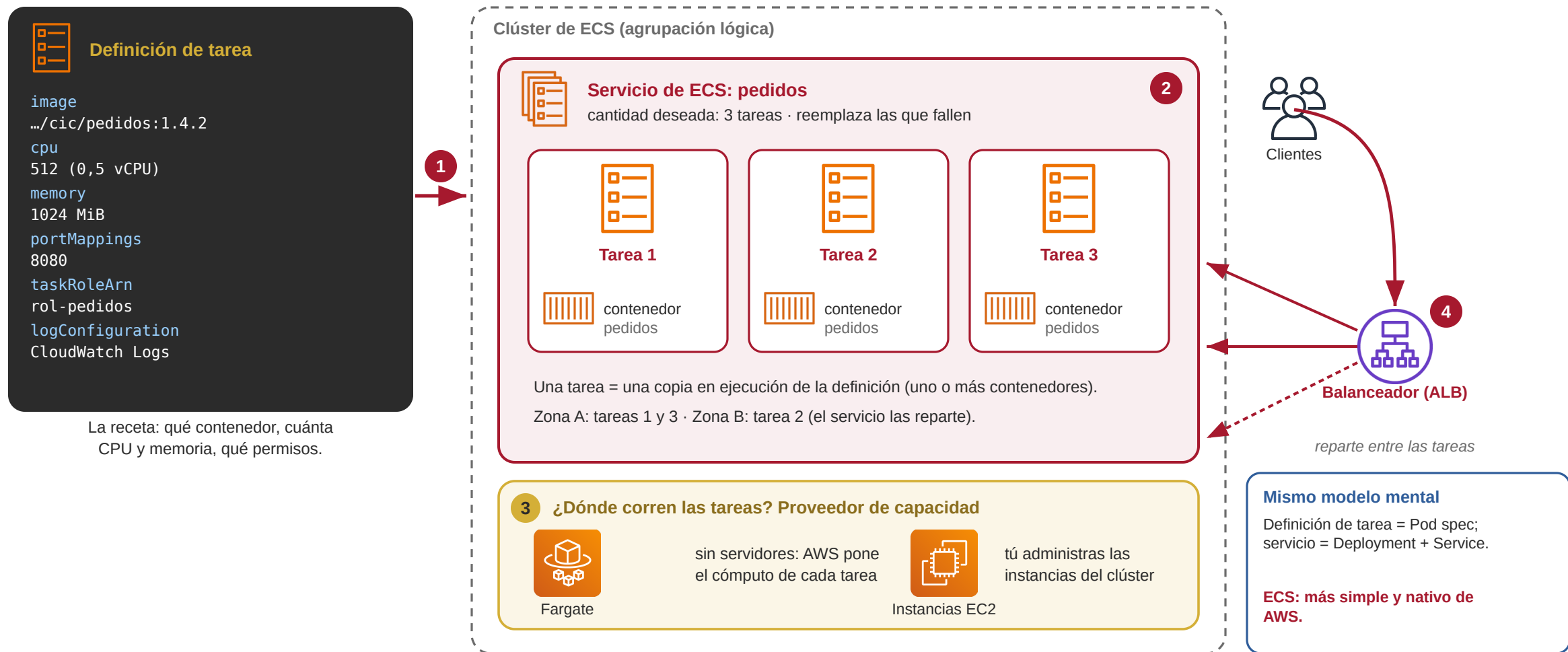
Pod, Deployment y Service: los objetos básicos

Declaras cuántas réplicas quieres; Kubernetes las crea, las reemplaza y les pone una dirección estable.



Amazon ECS: el orquestador nativo de AWS

Describes la tarea, pides cuántas copias y eliges quién pone el cómputo: Fargate o tus propias instancias.



Amazon EKS: Kubernetes administrado

AWS opera el plano de control; tú decides si los nodos son EC2 administrados o Fargate.



AWS Fargate: contenedores sin administrar servidores

Fargate no es otro orquestador: es una forma de ejecutar tareas de ECS o pods de EKS sin gestionar instancias.

ECS o EKS con instancias EC2		ECS o EKS con AWS Fargate	
Tu aplicación en un contenedor	TÚ	Tu aplicación en un contenedor	TÚ
Definir CPU y memoria de cada tarea	TÚ	Definir CPU y memoria de cada tarea	TÚ
Escalar el número de instancias del clúster	TÚ	Escalar el número de instancias del clúster	AWS
Sistema operativo, AMI y parches de seguridad	TÚ	Sistema operativo, AMI y parches de seguridad	AWS
Agente de contenedores y runtime	TÚ	Agente de contenedores y runtime	AWS
Servidor físico, red y centro de datos	AWS	Servidor físico, red y centro de datos	AWS
Más control, más trabajo operativo		Menos control, casi nada de operación	



Fargate
Cada tarea corre aislada en su propio entorno (microVM).

Se paga por vCPU y GB por segundo (mínimo un minuto).

Fargate es “contenedores sin servidores”: sigues usando ECS o EKS, pero ya no administras las instancias.
Ideal para cargas variables y equipos pequeños; con ocupación constante y alta, las instancias EC2 pueden salir más económicas.

¿Dónde ejecutar? Un espectro de control y operación

Cuanto más a la derecha, menos servidores administras y menos decisiones de bajo nivel puedes tomar.

Del máximo control a la mínima operación

más control

menos operación



Instancias EC2

Tú instalas Docker y administras todo.

Úsalo si...

Sistemas heredados o necesidades muy especiales.



ECS en EC2

Orquestador de AWS sobre tus instancias.

Úsalo si...

Muchos contenedores estables y de alto uso.



EKS

Kubernetes estándar con plano administrado.

Úsalo si...

Equipos que ya usan Kubernetes o quieren portabilidad.



ECS o EKS con Fargate

Contenedores sin nodos que administrar.

Úsalo si...

Microservicios con carga variable y poco personal.



Lambda

Solo código; el contenedor es opcional.

Úsalo si...

Eventos y tareas cortas: se ve en la Parte 3.

El mismo espectro existe en otras nubes



Máquinas virtuales



AKS



Container Apps



GKE



Cloud Run

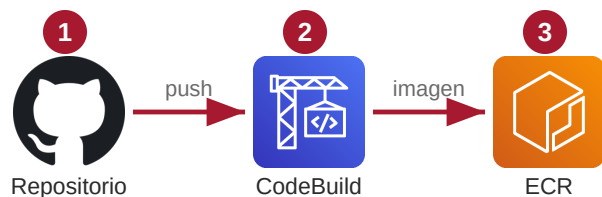


Azure Functions

Se elige por el nivel de control que el equipo necesita y la operación que puede asumir, no por moda.

Caso: Pedidos en contenedores sobre ECS y Fargate

Del repositorio a producción: cada cambio se convierte en una imagen versionada y en tareas detrás de un balanceador.

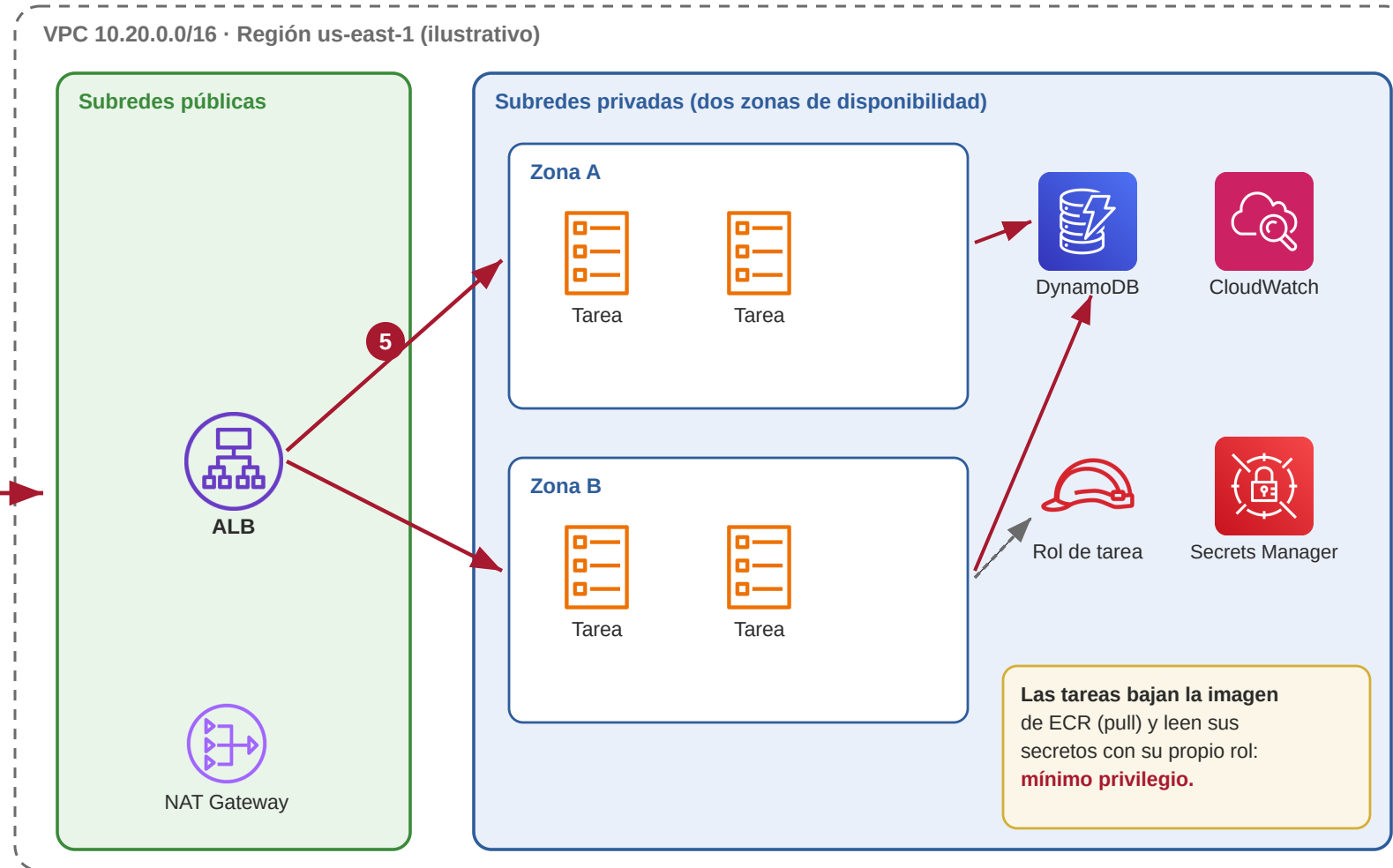


Cada cambio en la rama principal construye y publica una imagen nueva.



Lo que ganó la Corporación

- Mismo artefacto del portátil a producción.
- Despliegues por oleadas con reversa.
- Pedidos escala sin tocar otros servicios.



Lo que queda de la Parte 2

Empaquetar con imágenes, guardar en un registro, orquestar y decidir cuánto operar tú mismo.

1



Imagen y contenedor

La imagen es la receta en capas; el contenedor, un proceso aislado.

2



Registro

ECR guarda, versiona y escanea; producción usa solo imágenes propias.

3



Orquestar

Kubernetes, ECS y EKS programan, reparan, escalan y actualizan.

4



Fargate

Contenedores sin servidores: tú defines la tarea, AWS pone el cómputo.

Siguiente paso: ¿y si ni siquiera quiero pensar en contenedores?



Servidores



Contenedores



Fargate



Funciones

Parte 3: serverless y FaaS.

Lambda, API Gateway, DynamoDB y Step Functions.

Serverless y FaaS: Lambda, API Gateway, DynamoDB y Step Functions

Parte 3 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Ejecutar código por evento, exponerlo como API y orquestarlo sin administrar servidores

Serverless: no es que no haya servidores, es que no son tuyos

Cuanto más a la derecha, menos capas administras: en FaaS solo te queda el código.

Lo que administras en cada modelo (vino: tú · dorado: el proveedor)

Código de la aplicación	tú	tú	tú	tú
Escalado y capacidad	tú	tú	tú	AWS
Entorno de ejecución	tú	tú	tú	AWS
Sistema operativo	tú	tú	AWS	AWS
Virtualización	tú	AWS	AWS	AWS
Hardware y red	tú	AWS	AWS	AWS
	On-premises	IaaS (EC2)	Contenedores en Fargate	FaaS (Lambda)



Sin servidores que administrar

Provisión, parches y capacidad quedan del lado del proveedor.



Escala automática

Desde cero hasta miles de ejecuciones, sin configurar el escalado.



Pago por uso real

Si nadie usa la aplicación, casi no pagas.



Alta disponibilidad incluida

El servicio ya viene repartido en varias zonas.

Servicios serverless de AWS que usaremos esta semana:

 **Lambda**

 **API Gateway**

 **DynamoDB**

 **Step Functions**

 **S3**

 **SQS**

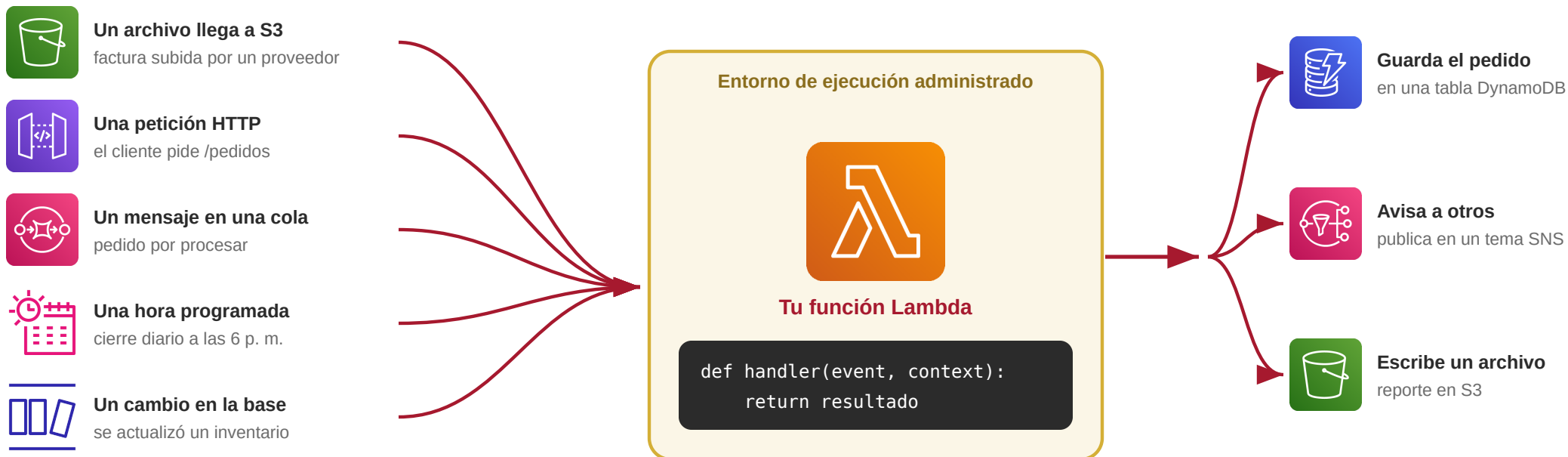
 **EventBridge**

 **Cognito**

FaaS: del evento a la función y al resultado

Escribes una función; la nube la ejecuta cada vez que ocurre un evento y la apaga cuando termina.

Eventos: lo que dispara la función



Invocación síncrona

API Gateway llama y espera la respuesta.

Invocación asíncrona

S3, SNS y EventBridge entregan el evento y siguen; Lambda reintenta si falla.

Invocación por sondeo

Lambda lee SQS, Kinesis o DynamoDB Streams y entrega lotes.

Por dentro de Lambda: Init, Invoke y Shutdown

El código fuera del handler corre una vez por entorno; el handler corre en cada evento.

```
import json, boto3

tabla = boto3.resource("dynamodb").Table("Pedidos")

def handler(event, context):
    pedido = json.loads(event["body"])
    tabla.put_item(Item=pedido)
    return {"statusCode": 201}
```

Init: una sola vez

Invoke: en cada evento

Lo que va fuera del handler (conexiones, clientes del SDK) se reutiliza mientras el entorno siga vivo; lo que va dentro se repite siempre.

Vida de un entorno de ejecución

Init

Invoke 1

Invoke 2

Invoke 3

Shutdown

Una sola invocación a la vez por entorno; si llegan más a la vez, Lambda crea otros entornos.

Dentro de Init

- 1 Descargar el código o la imagen
- 2 Iniciar el entorno de ejecución y el runtime
- 3 Ejecutar el código fuera del handler

Después, el entorno queda “caliente”: las siguientes invocaciones saltan Init y empiezan en Invoke.

Shutdown ocurre cuando Lambda recicla el entorno.

Init

Prepara el entorno una sola vez: descarga el código e inicia el runtime.

Invoke

Ejecuta tu handler con el evento; aquí se cobra la duración.

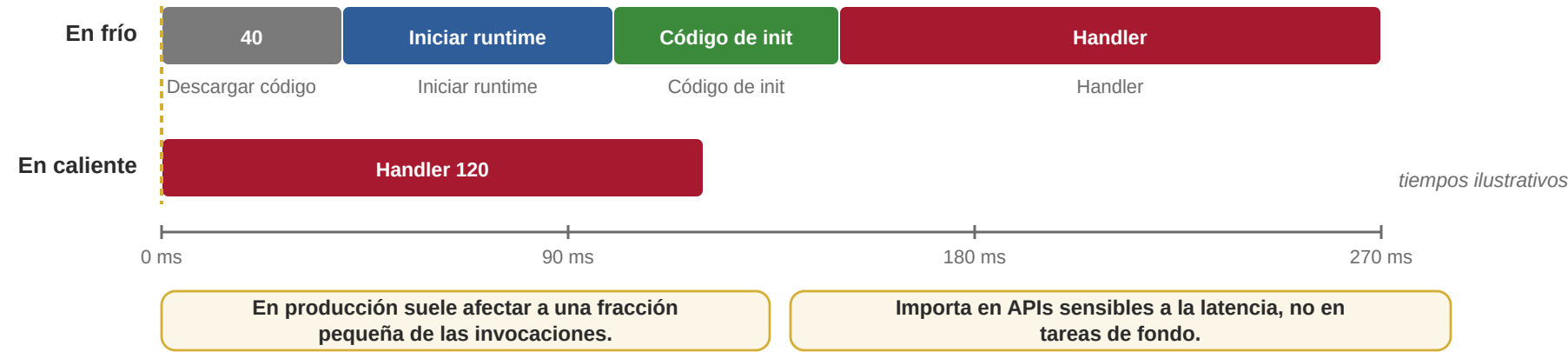
Shutdown

Libera el entorno cuando Lambda lo decide; no lo controlas tú.

Arranque en frío frente a arranque en caliente

El frío agrega la preparación del entorno a la primera invocación; el caliente va directo al handler.

Arranque en frío: la primera invocación de un entorno nuevo



Cómo reducirlo

Provisioned Concurrency

Mantiene entornos ya inicializados y listos; se paga por tenerlos.

SnapStart

Toma una instantánea del entorno inicializado y la restaura rápido (Java, Python, .NET).

Paquete pequeño

Menos dependencias y solo los módulos que se usan.

Memoria y arquitectura

Más memoria da más CPU; arm64 (Graviton) mejora precio-rendimiento.

Concurrencia y escalado: un entorno por solicitud simultánea

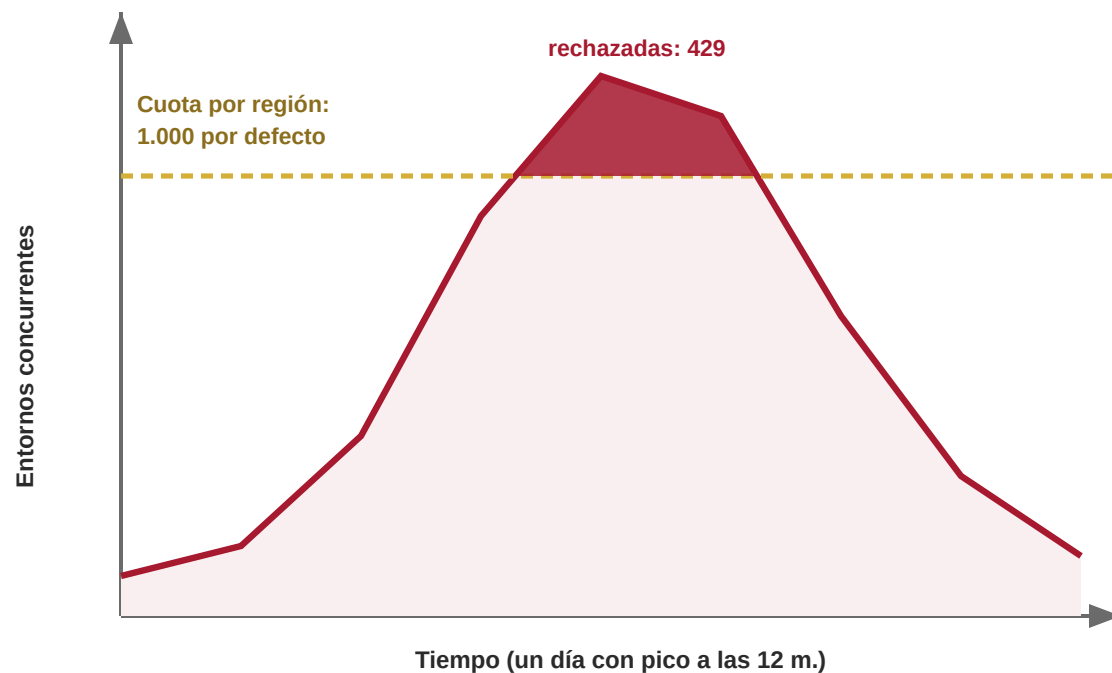
Lambda escala horizontalmente por sí sola, pero la cuota regional pone un techo que hay que conocer.

Concurrencia = solicitudes por segundo × duración (s)

Ejemplo: 100 solicitudes por segundo × 0,5 s = 50 entornos activos a la vez.

Cada solicitud simultánea usa su propio entorno de ejecución.

Si baja la demanda, los entornos sobrantes se retiran.



Escala por sí sola

Lambda agrega entornos a un ritmo limitado por función; ante un pico muy brusco puede haber rechazos.



Reserved concurrency

Garantiza capacidad a una función y la limita para que no consuma toda la cuota.



Provisioned concurrency

Mantiene entornos listos de antemano para picos previsibles.

Límites de Lambda: lo que no puede hacer una función

Una función es corta, liviana y sin estado; lo largo o pesado va a otro servicio.

**15 min****Tiempo máximo por invocación (900 segundos)**

Lo que dura más se parte o se mueve a otro servicio.

**128 MB a 10.240 MB****Memoria configurable**

La CPU crece con la memoria, hasta 6 vCPU.

**6 MB****Carga máxima síncrona**

Entrada y salida de una invocación síncrona.

**512 MB****Almacenamiento temporal /tmp**

Configurable hasta 10.240 MB; se pierde con el entorno.

**50 MB · 10 GB****Paquete zip · imagen de contenedor**

El zip admite hasta 250 MB descomprimido.

**1.000****Ejecuciones simultáneas por región**

Valor por defecto; ajustable pidiéndolo a AWS.

¿Tu tarea supera un límite? Cambia de herramienta, no de modelo mental

**Fargate**

procesos largos o con mucha memoria

**AWS Batch**

cálculo por lotes de horas

**Step Functions**

partir un proceso largo en pasos

Los valores son los vigentes en la documentación de AWS Lambda; confirma siempre las cuotas actuales de tu región y cuenta.

Precios: pagas por invocación y por GB-segundo

Sin tráfico casi no hay factura; con carga alta y constante conviene comparar con servidores.

Cómo se calcula la factura de Lambda

Solicitudes
0,20 USD por
millón de invocaciones

+

Duración × memoria
0,0000166667 USD por
GB-segundo (x86, us-east-1)

Nivel gratuito mensual: 1 millón de solicitudes y 400.000 GB-segundo

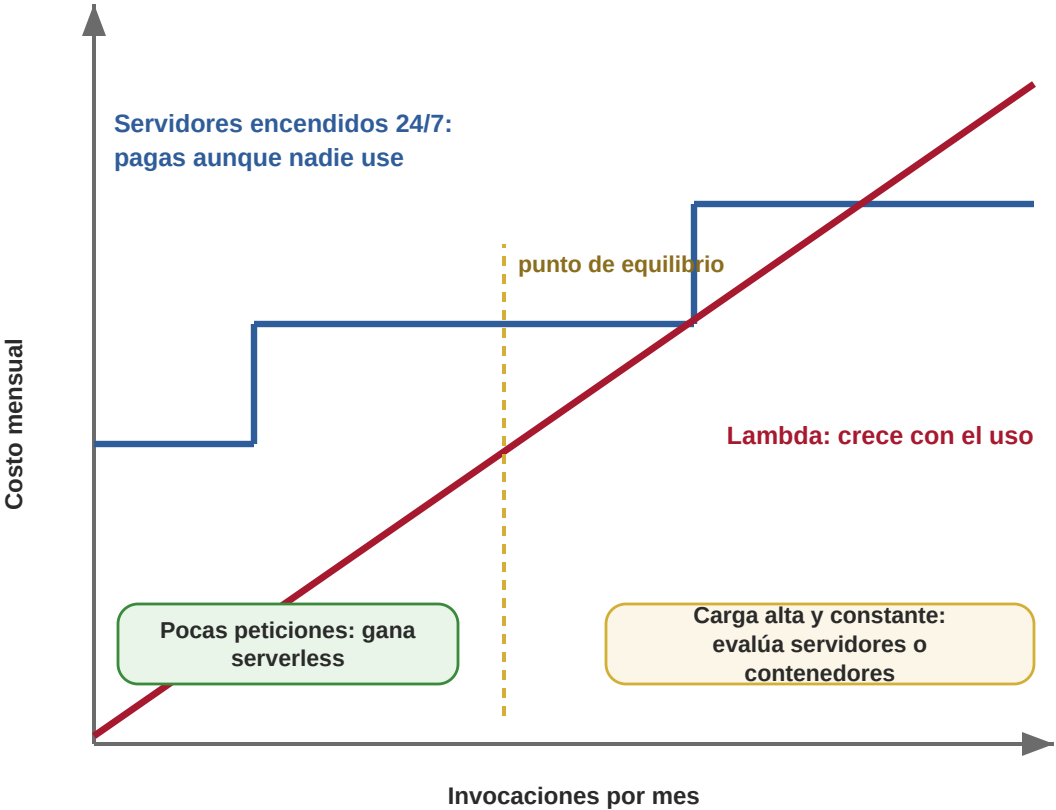


Ejemplo de la Corporación (ilustrativo)

Pedidos procesados al mes	3.000.000
Duración de cada ejecución	200 ms (0,2 s)
Memoria configurada	512 MB (0,5 GB)
GB-segundo = 3.000.000 × 0,2 × 0,5	300.000
Cobro por duración	0 USD (dentro de 400.000 gratis)
Cobro por solicitudes (3 M – 1 M) × 0,20	0,40 USD

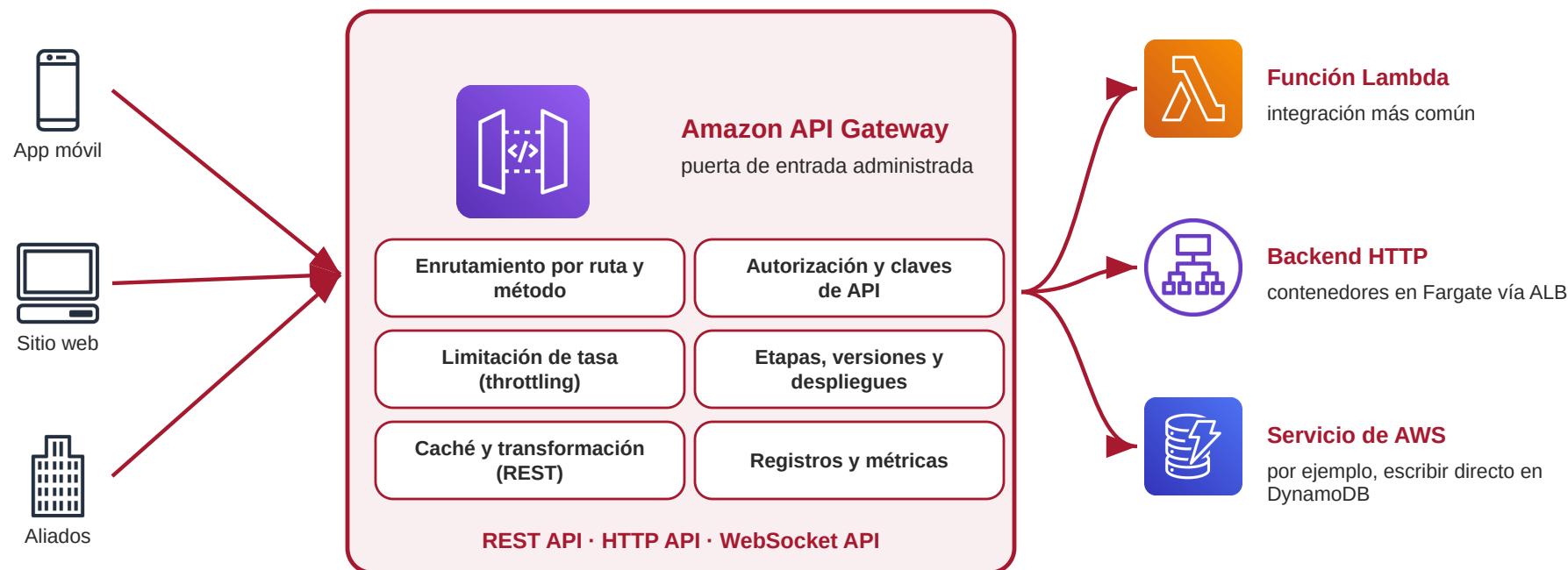
Factura mensual aproximada: menos de 1 dólar

Costo mensual frente al uso (curvas ilustrativas)



API Gateway: la puerta de entrada HTTP de tus funciones

Un servicio administrado que recibe las peticiones, las valida y las entrega a la integración correcta.



REST API

- Caché integrada
- Validación de solicitudes
- Claves de API y planes de uso
- Compatible con AWS WAF

HTTP API

- Más simple y más barata
- Autorizador JWT integrado
- Menor latencia
- Ideal para funciones y backends

WebSocket API

- Conexión persistente
- Mensajes en ambos sentidos
- Chat y paneles en vivo
- Rutas por tipo de mensaje

Etapas, despliegues y limitación de tasa

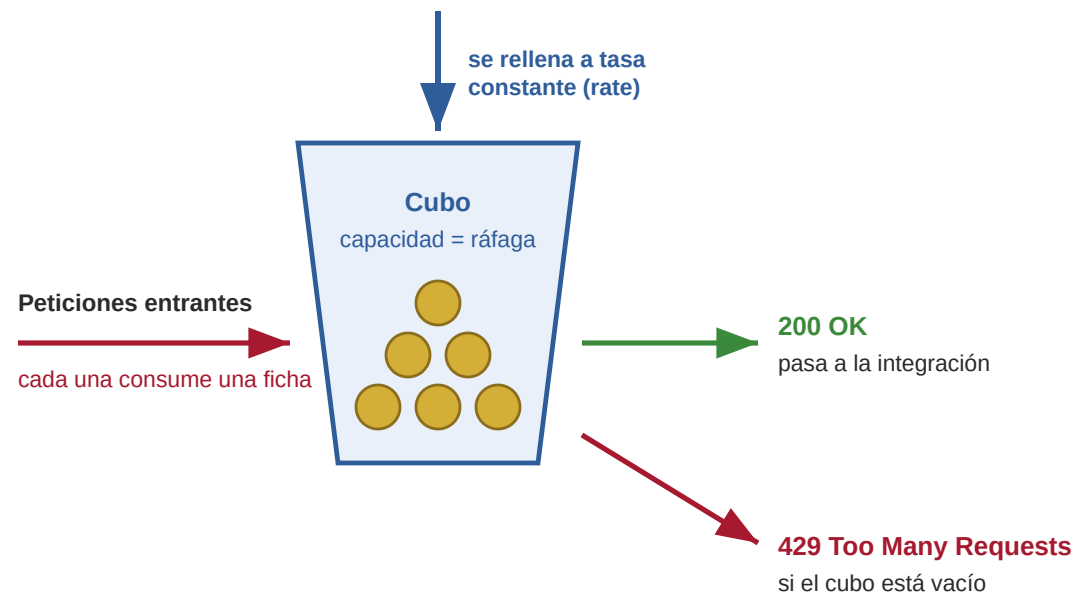
Una API se publica en etapas con su propia URL y se protege con un cubo de fichas que corta los excesos.

Etapas: la misma API en distintos entornos



Cada etapa tiene su propia URL, variables, registros y límites.

Limitación de tasa: el cubo de fichas



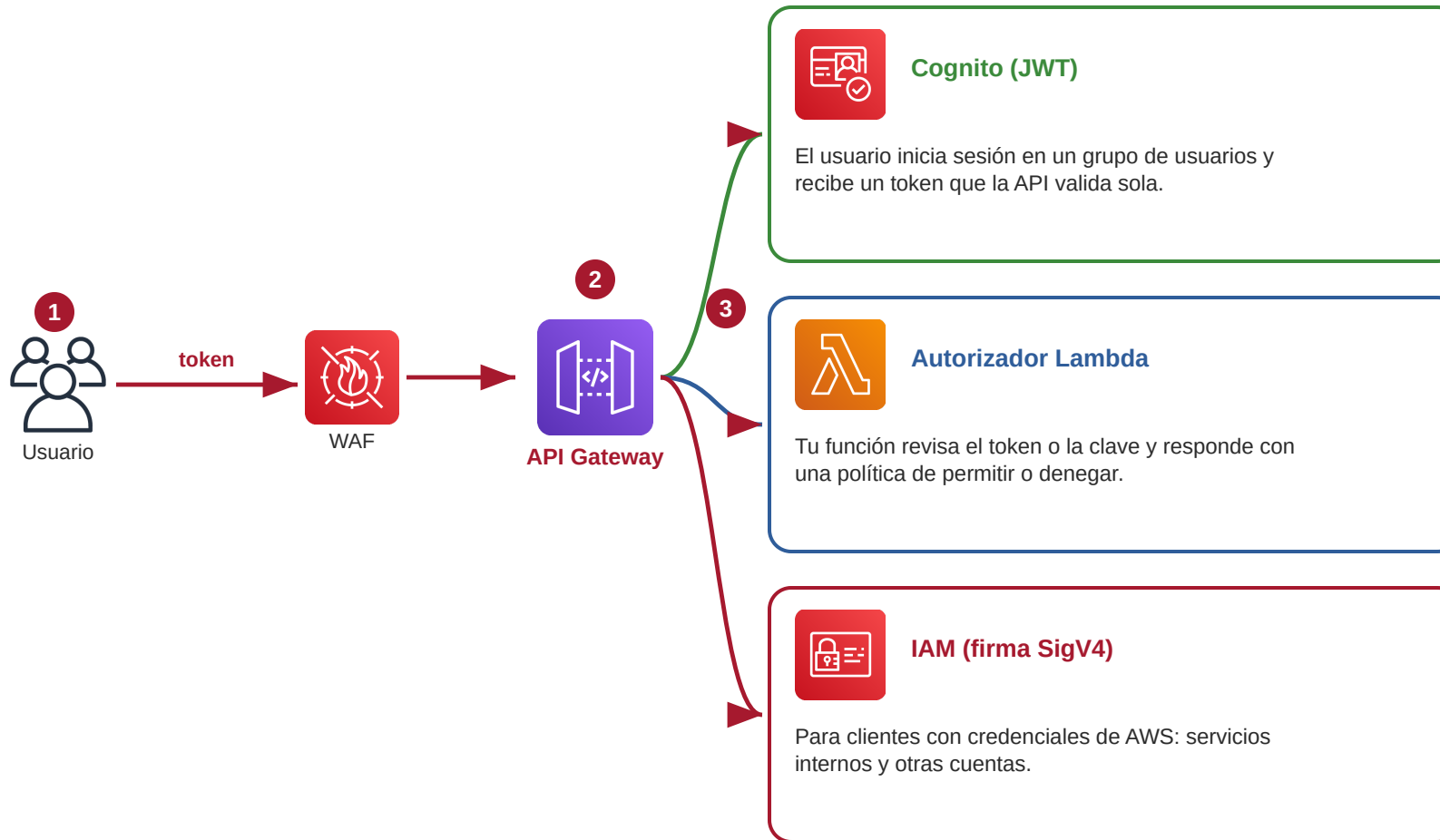
Límites por defecto (por cuenta y región)

10.000 solicitudes por segundo y ráfaga de 5.000. Se pueden fijar límites más bajos por etapa, por método y por cliente con planes de uso.

Protege tu backend y tu factura de picos y abusos.

Autorización en API Gateway: Cognito, Lambda o IAM

La API verifica quién eres antes de despertar a la función; así los intrusos no generan costo ni riesgo.



Si el token es válido



Tu función

4



DynamoDB

Si no lo es

401 No autorizado · 403 Prohibido

Buenas prácticas

- Valida en la API antes de invocar la función.
- Un rol de IAM distinto por función.
- AWS WAF delante bloquea ataques comunes.

DynamoDB: la base de datos que escala con Lambda

Clave de partición para repartir, clave de orden para organizar: sin servidores, sin conexiones que agotar.

Tabla Pedidos (modelo clave-valor y documento)

clienteld · PK	pedidoId · SK	estado	total
CLI-014	P-0001	pagado	1.250.000
CLI-014	P-0002	en camino	480.000
CLI-077	P-0003	pagado	3.900.000
CLI-102	P-0004	pendiente	150.000

Cada fila es un ítem; los atributos pueden variar entre ítems.

Cómo escala



DynamoDB aplica una función hash a la clave de partición y reparte los ítems entre particiones en varias zonas.

Partición 1

hash (CLI-014)

ítems con esa clave

Partición 2

hash (CLI-077)

ítems con esa clave

Partición 3

hash (CLI-102)

ítems con esa clave

Dos modos de capacidad

Bajo demanda: pagas por lectura y escritura

Aprovisionada: fijas capacidad, con autoescalado

Latencia de milisegundos de un solo dígito a cualquier escala.
Replica automáticamente en tres zonas de disponibilidad.



Streams

cada cambio dispara una función



Índices secundarios

consultar por otros atributos



DAX

caché en memoria para lecturas



TTL y tablas globales

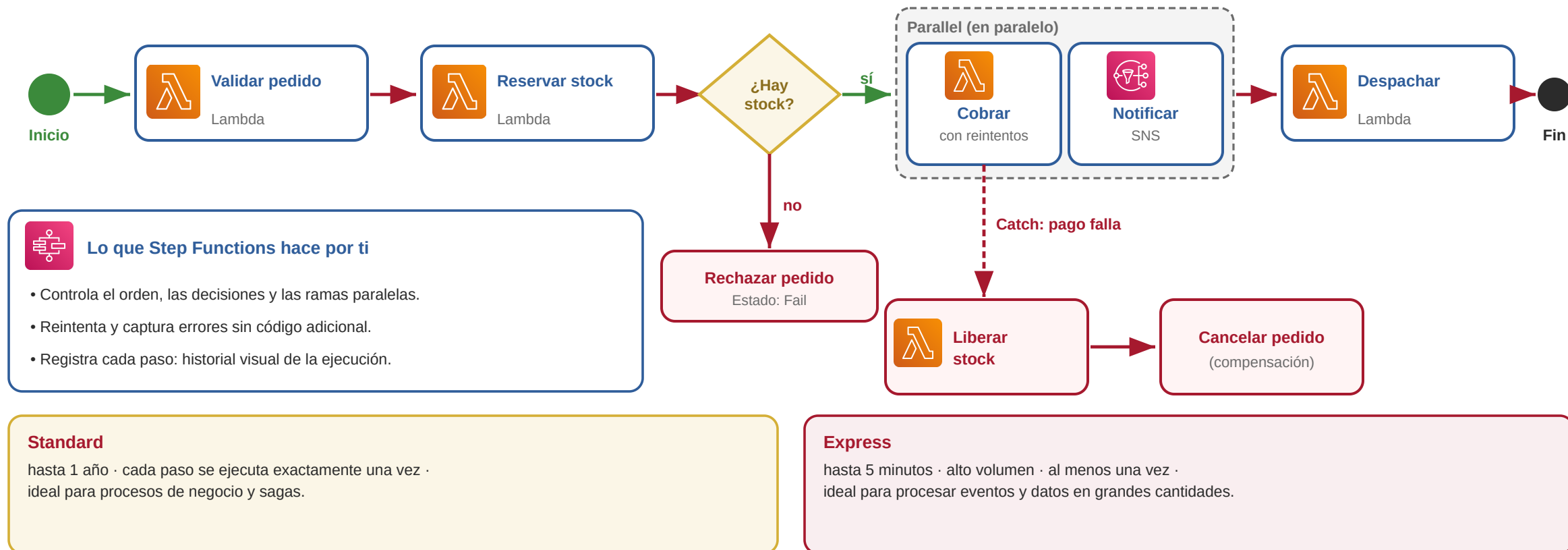
expiración y réplica multirregión

Diseña la tabla desde las consultas que necesitas.

Step Functions: orquestar funciones como un flujo de estados

La saga del pedido se dibuja como máquina de estados: pasos, decisiones, paralelo y compensación.

Máquina de estados del pedido (flujo de trabajo estándar)



Ejemplo en Azure: Functions con disparadores y enlaces

El modelo es el mismo: un evento dispara la función, y los enlaces de salida escriben el resultado.

Disparadores (triggers)



Blob Storage

llega un archivo



Event Grid

evento de un servicio



Service Bus

mensaje en cola o tema



HTTP

petición web

Temporizador

hora programada (cron)



Azure Function App

una o varias funciones que comparten plan y configuración

```
[Function("Pedidos")]
public static void Run(
    [BlobTrigger] Stream blob,
    [CosmosDBOutput] ...)
```

Salidas (output bindings)



Cosmos DB

guarda el documento



Queue Storage

deja un mensaje



Blob Storage

escribe un archivo

Plan Flex Consumption

escala a cero y paga por uso; recomendado para empezar

Plan Premium

instancias precalentadas, sin arranque en frío

Plan Dedicado

sobre un plan de App Service ya existente

Durable Functions: flujos con estado en código, equivalente a Step Functions · Google Cloud: Cloud Run functions

Lo que queda de la Parte 3

Serverless desplaza la operación al proveedor; a cambio, el diseño debe respetar sus límites.

1



FaaS

Evento, función y resultado;
Lambda crea, ejecuta y apaga el
entorno.

2



Frío y límites

Init fuera del handler; 15
minutos, 6 MB y cuotas de
concurrency.

3



API y datos

API Gateway autoriza y limita;
DynamoDB escala sin servidores.

4



Orquestar

Step Functions coordina pasos,
paralelo y compensaciones.

Siguiente paso: juntar las piezas en una aplicación completa y llevarla al laboratorio



S3



CloudFront



API Gateway



Lambda



DynamoDB

Arquitectura serverless de referencia y laboratorio

Parte 4 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

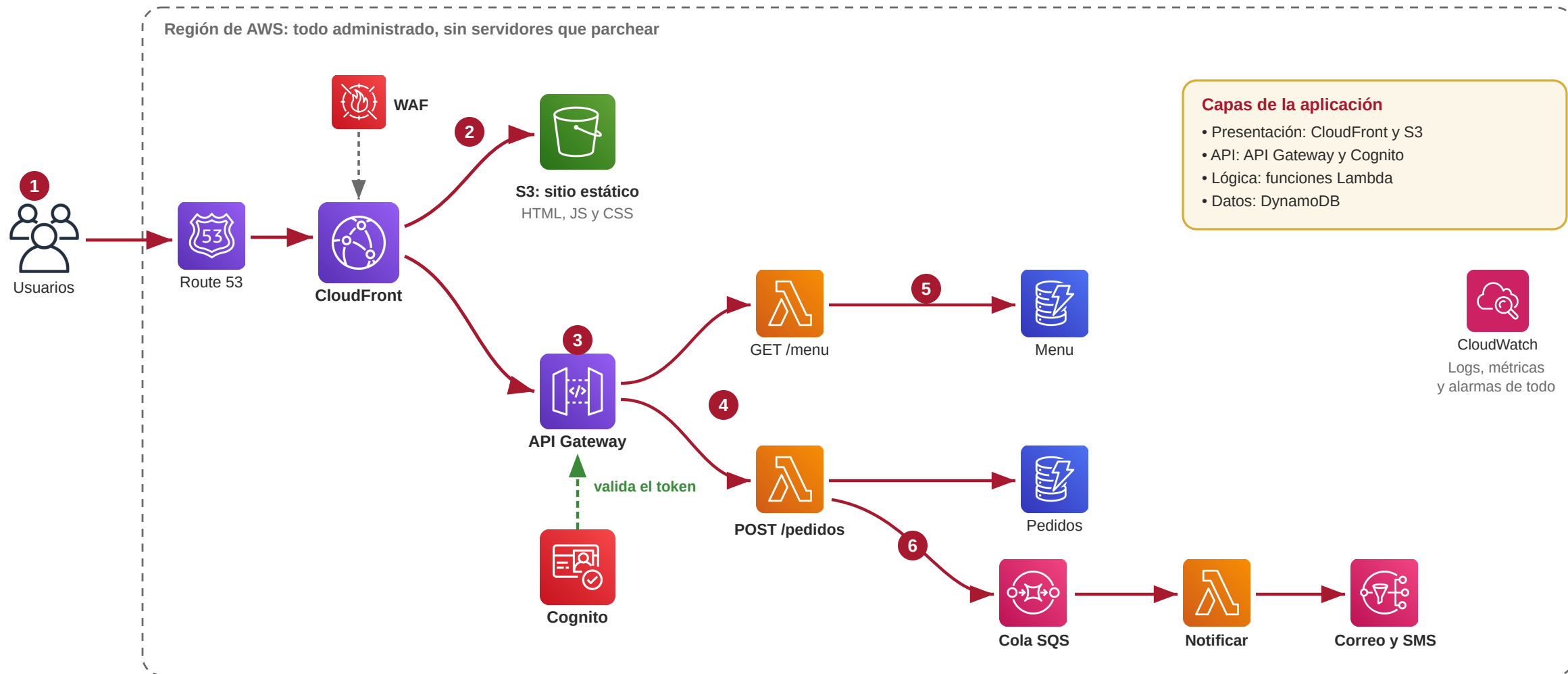
Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Una aplicación web completa sin servidores, sus buenas prácticas y el laboratorio de la cafetería

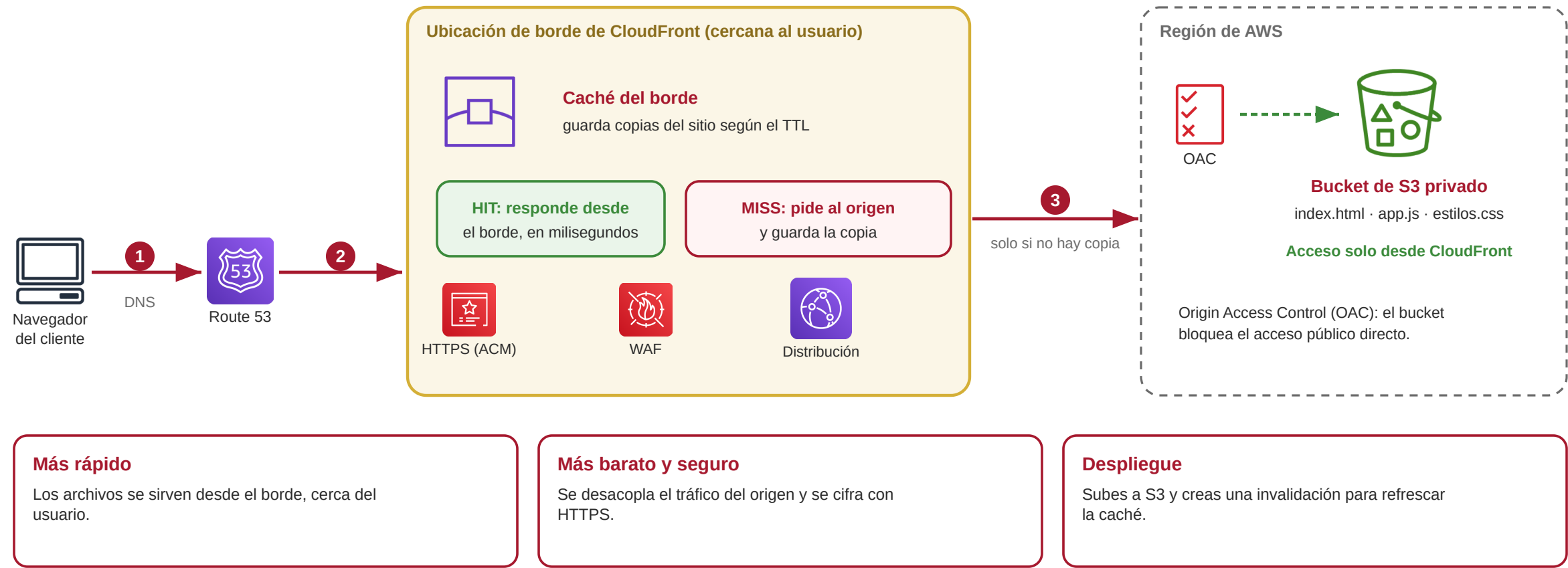
Aplicación web serverless de referencia

Cinco servicios administrados forman una aplicación completa: ningún servidor que parchear ni escalar.



Presentación: S3 y CloudFront

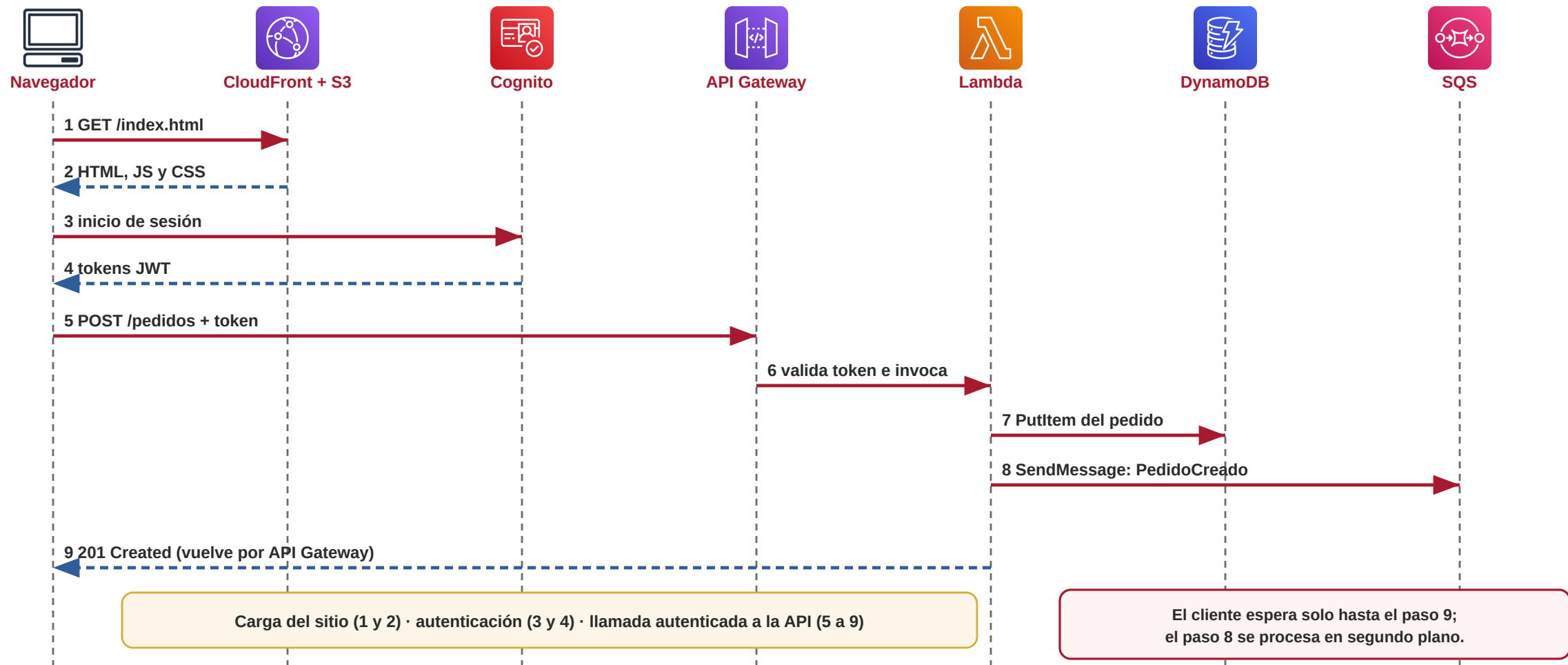
El sitio es un conjunto de archivos estáticos; CloudFront los acerca al usuario y protege al bucket.



Mismo patrón que el sitio estático de la Semana 3 y la caché de la Semana 12, ahora como puerta de una aplicación.

Una petición de punta a punta

Nueve mensajes entre siete componentes: desde abrir el sitio hasta registrar y notificar el pedido.



Cuatro patrones serverless que se repiten

Casi toda aplicación serverless combina estas cuatro formas de disparar y encadenar funciones.

API web síncrona

El cliente espera la respuesta.

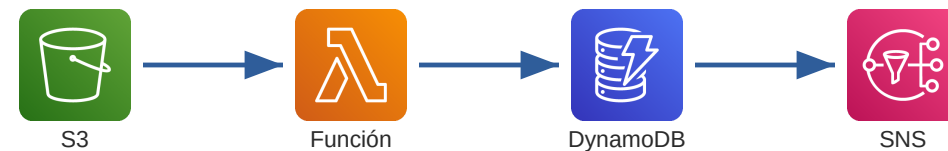


Ejemplo:

Consultar el menú, registrar un pedido.

Procesar archivos

Un archivo nuevo dispara el trabajo.

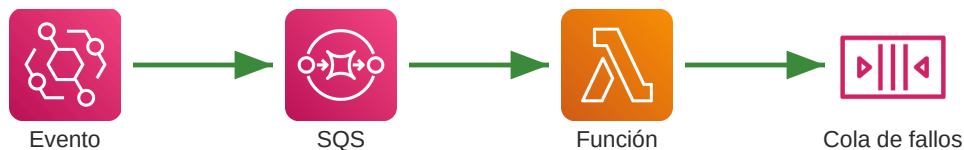


Ejemplo:

Una factura PDF llega y se extraen sus datos.

Cola con consumidores

La cola amortigua los picos.

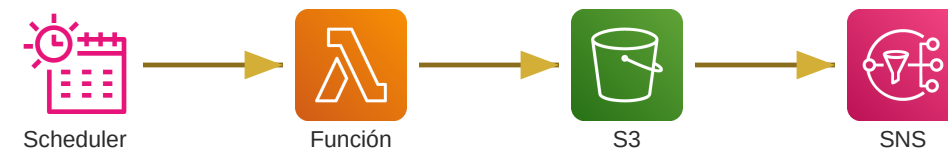


Ejemplo:

Mil pedidos en una hora pico se procesan a ritmo constante.

Tarea programada

El reloj es el evento.



Ejemplo:

Cada noche se consolida el reporte de ventas.

Seguridad: capas y mínimo privilegio por función

En serverless el perímetro es la identidad: cada función recibe solo el permiso que necesita.

Defensa en capas: cada una verifica algo distinto



Mínimo privilegio por función (política de IAM)

```
{
  "Effect": "Allow",
  "Action": "dynamodb:PutItem",
  "Resource": "arn:aws:dynamodb:us-east-1:
    123456789012:table/Pedidos"
}
```

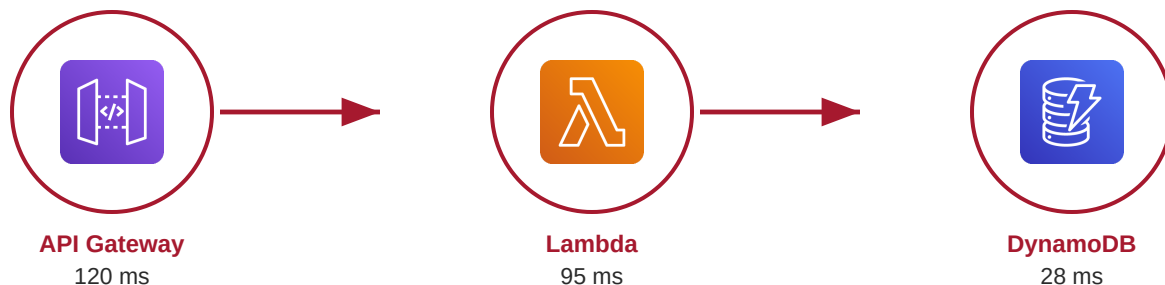
Por qué importa

- 1** La función de pedidos solo puede escribir en Pedidos.
- 2** Si alguien la compromete, no puede leer Menu ni borrar tablas.
- 3** Evita el rol único con permisos de administrador.
- 4** Secretos fuera del código: Secrets Manager o Parameter Store.

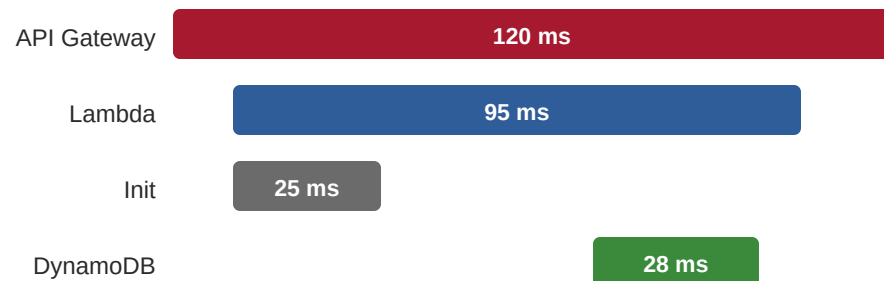
Observabilidad: logs, métricas y trazas

No puedes arreglar lo que no ves: tres señales te dicen qué pasó, cuánto y dónde.

Mapa de servicios y trazas (AWS X-Ray)

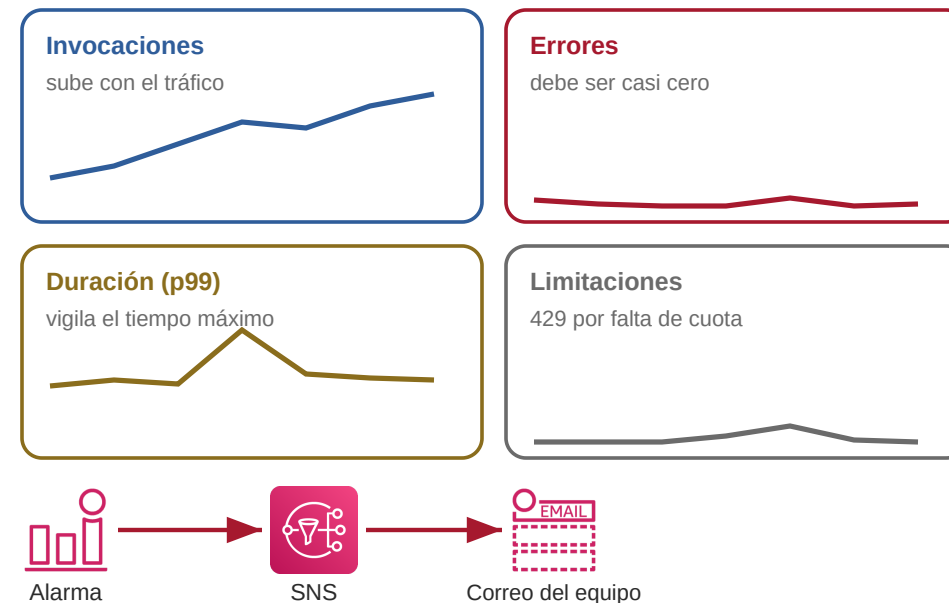


Línea de tiempo de una solicitud



Ejemplo ilustrativo: la base de datos es solo una parte del tiempo total.

Métricas de CloudWatch por función

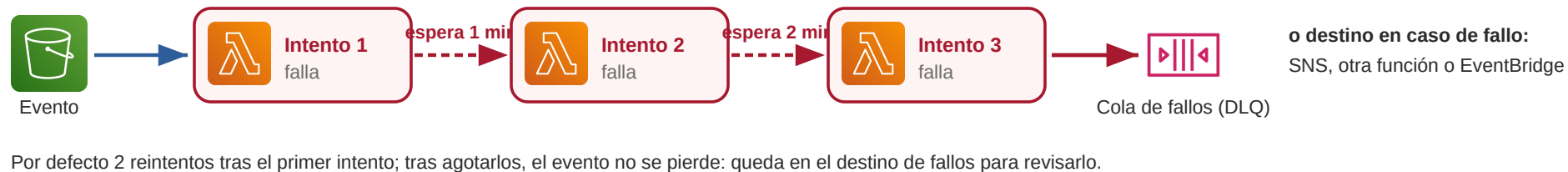


Registros: cada función escribe en CloudWatch Logs; usa logs en formato JSON con un identificador de correlación (por ejemplo el pedidoId) para seguir un pedido a través de API Gateway, Lambda y DynamoDB.

Resiliencia: reintentos, cola de fallos e idempotencia

Los fallos ocurren; el diseño decide si se reintenta, se aparta o se ignora un duplicado.

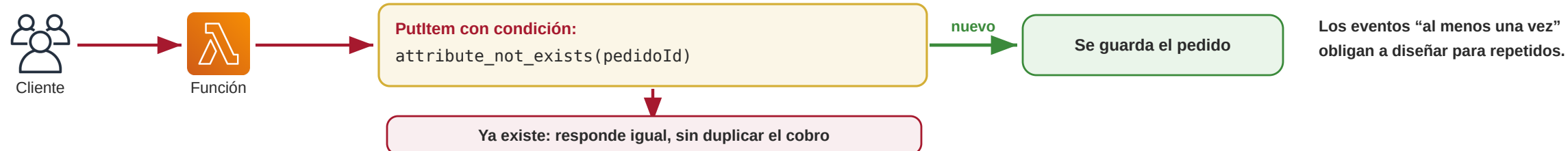
Invocación asíncrona: Lambda reintenta por ti



Cola SQS como fuente: el mensaje vuelve si la función falla



Idempotencia: el mismo pedido puede llegar dos veces, y debe contarse una

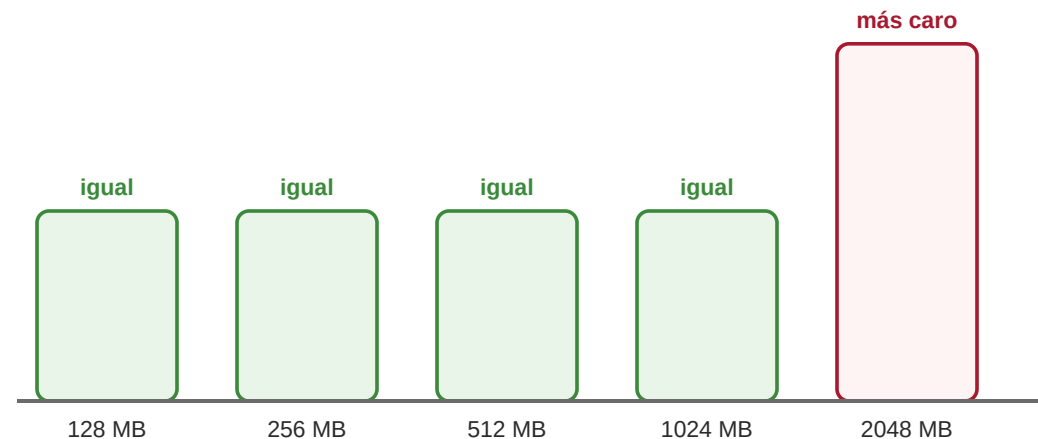
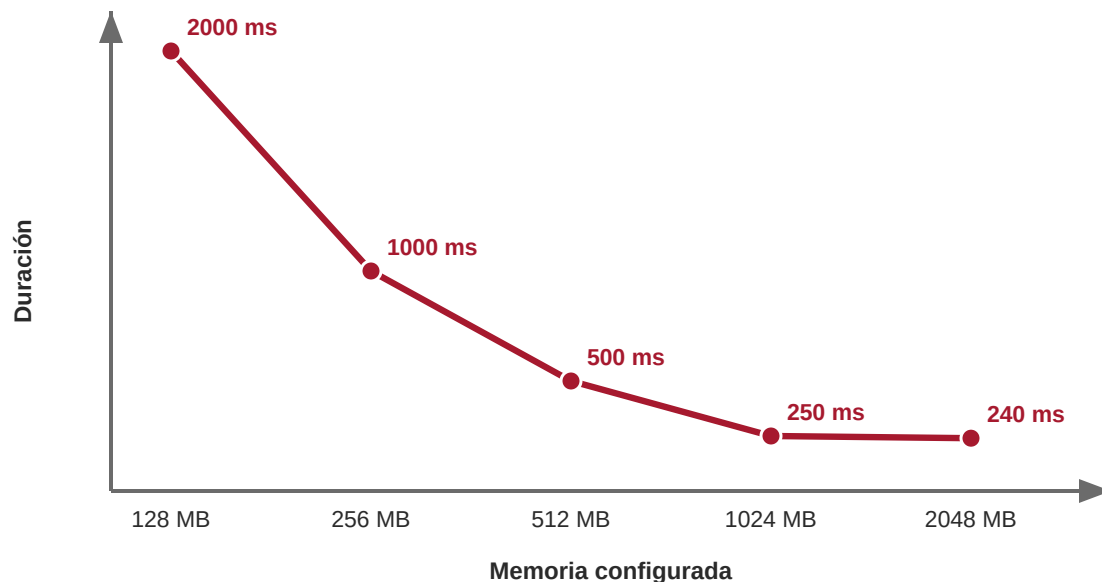


Costo y rendimiento: afinar memoria, arquitectura y caché

En Lambda, más memoria a veces cuesta lo mismo y responde mucho más rápido: se mide, no se adivina.

Más memoria = más CPU: la duración baja (ilustrativo, función intensiva en CPU)

Costo por invocación (memoria × duración)



Hasta 1.024 MB cuesta lo mismo y responde 8 veces más rápido.



Afina la memoria

AWS Lambda Power Tuning o Compute Optimizer miden varias configuraciones y sugieren la mejor.



Usa arm64 (Graviton)

AWS indica hasta 34 % mejor relación precio-rendimiento; valida tu carga.



Cachea y reutiliza

CloudFront, caché de API y conexiones fuera del handler evitan trabajo repetido.



Controla el ruido

Retención de logs acotada y logs de nivel adecuado para no pagar de más.

Well-Architected: los seis pilares aplicados a serverless

Tres prácticas por pilar resumen lo que revisaríamos en cualquier arquitectura serverless de la Corporación.



Excelencia operativa

- 1 Logs JSON y trazas con X-Ray
- 2 Infraestructura como código (SAM o CloudFormation)
- 3 Despliegues canario con reversa



Seguridad

- 1 Un rol mínimo por función
- 2 Autorizar en la API antes de invocar
- 3 Secretos en Secrets Manager



Fiabilidad

- 1 Reintentos y cola de fallos
- 2 Idempotencia en las escrituras
- 3 Límites de concurrencia y de tasa



Eficiencia de desempeño

- 1 Memoria ajustada con medición
- 2 arm64 y paquetes pequeños
- 3 Caché en CloudFront y API Gateway



Optimización de costos

- 1 Pago por uso real
- 2 Evitar sobreaprovisionar memoria
- 3 Etiquetas y presupuestos por servicio



Sostenibilidad

- 1 Escalar a cero cuando no hay uso
- 2 Aprovechar capacidad compartida
- 3 Menos recursos ociosos que mantener

Caso integrador: elegir el cómputo para cada servicio

No hay una respuesta única: cada componente se ubica según su carga, su duración y su equipo.

Caso integrador: ¿qué cómputo para cada parte de la Corporación Industrial del Caribe?

Componente	Elección	Por qué
 Sitio web de clientes	 S3 + CloudFront	Contenido estático: sin cómputo que administrar.
 Pedidos	 ECS en Fargate	Tráfico constante, lógica extensa y equipo con experiencia en contenedores.
 Inventario	 Lambda + RDS Proxy	Cargas cortas por evento; el proxy cuida las conexiones a la base relacional.
 Facturación DIAN	 ECS en Fargate	Librerías pesadas y procesos más largos que 15 minutos.
 Notificaciones	 Lambda + SNS	Eventos esporádicos: pagas solo cuando se envían.
 Reportes nocturnos	 Scheduler + Lambda	Tarea programada que consulta y guarda el resultado en S3.

Regla práctica: empieza por lo más administrado y baja a contenedores solo cuando necesites control.

Laboratorio de la cafetería: arquitectura objetivo

Una API recibe pedidos, una función los guarda, otra avisa al cliente y un reloj consolida las ventas.

10 % de la nota · Laboratorio de AWS: arquitectura serverless de la cafetería



Recursos del lab

- 1 tabla DynamoDB
- 2 o 3 funciones Lambda
- 1 API en API Gateway
- 1 tema SNS y suscripción
- 1 regla de Scheduler
- Logs en CloudWatch



Antes de empezar

- Sigue el enunciado oficial del laboratorio en AWS Academy: este mapa es para que entiendas qué estás construyendo y por qué.
- Trabaja en la región que indique el laboratorio y, si no puedes crear roles de IAM, usa el rol que provea el entorno (por ejemplo LabRole).
- Nombra los recursos con tu código o apellido para identificarlos y borrarlos luego.

El laboratorio paso a paso

Siete pasos, de los datos hacia afuera, probando cada pieza antes de conectar la siguiente.

1 Crear la tabla



DynamoDB, tabla Pedidos con clave de partición pedidoId (texto), modo bajo demanda.

2 Crear registrarPedido



Python, rol de ejecución con permiso PutItem sobre la tabla. Aumenta el tiempo límite a 10 segundos.

3 Probar la función



Evento de prueba con un pedido JSON; comprueba que aparece el ítem en la tabla.

4 Crear la API



API HTTP, ruta POST /pedidos con integración a la función; copia la URL de invocación.

5 Crear el aviso



Tema SNS y suscripción por correo: confirma el mensaje que llega a tu bandeja.

6 Programar el reporte



Scheduler cada noche que invoca reporteDiario y guarda el resultado en S3.

7 Verificar y limpiar



Revisa los logs, captura las evidencias y elimina todos los recursos al terminar.

Prueba desde la terminal

```
curl -X POST \  
-d '{"pedidoId":"P-001",  
      "cafe":"tinto","cant":2}' \  
https://<id>.execute-api.  
us-east-1.amazonaws.com/pedidos
```

Esperado: respuesta 201.

Orden recomendado: construye de adentro hacia afuera

Datos: tabla

Lógica: función

Entrada: API

Eventos: aviso y reloj

Control: logs y limpieza

Prueba cada pieza antes de conectar la siguiente: así sabes dónde falló.

Verificar, depurar y limpiar

Entrega evidencias, resuelve los cinco errores típicos y borra todo para no dejar costos ocultos.



Evidencias a entregar

- 1 Tabla Pedidos con al menos un ítem.
- 2 Respuesta 201 de la API en la terminal.
- 3 Correo de confirmación del tema SNS.
- 4 Registro de la función en CloudWatch Logs.
- 5 Diagrama con tu nombre y los recursos creados.



Errores frecuentes

- 1 Error 500 o 502: abre CloudWatch Logs de la función.
- 2 AccessDenied: al rol le falta el permiso sobre la tabla.
- 3 Timeout: el tiempo límite por defecto es de 3 segundos.
- 4 No llega el correo: falta confirmar la suscripción.
- 5 Error de CORS: habilita CORS en la API (si hay navegador).



Limpeza para no gastar

- 1 Eliminar la API de API Gateway.
- 2 Eliminar las funciones Lambda.
- 3 Eliminar la tabla de DynamoDB.
- 4 Eliminar el tema SNS y las suscripciones.
- 5 Eliminar la regla de Scheduler, los grupos de logs y el bucket.

Recuerda

Lambda y API Gateway cobran por uso y el costo del laboratorio es mínimo, pero los recursos olvidados, como tablas aprovisionadas, buckets llenos o logs sin retención, pueden seguir cobrando.

Semana 14 en cuatro ideas

De monolito a servicios, de servidores a contenedores y de contenedores a funciones.

1 Microservicios



Se corta por dominio; cada servicio tiene sus datos y se comunica con cuidado.

2 Contenedores



Imagen, registro y orquestador; Fargate quita los servidores.

3 Serverless y FaaS



Evento, función y resultado; pagas por uso y respetas los límites.

4 Aplicación completa

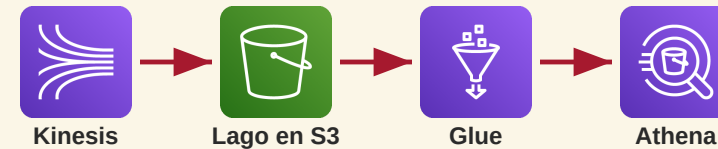


S3, CloudFront, API Gateway, Lambda y DynamoDB, con las buenas prácticas.

Próxima semana

Semana 15 · Patrones de ingeniería de datos

Las 5 V de los datos, ingesta por lotes y en flujo, lago de datos, catálogo y analítica.



Los pedidos y eventos de esta semana serán la materia prima de esos datos.



Pendiente para ti

Entrega el laboratorio de la cafetería con sus evidencias (10 % de la nota) y repasa los límites de Lambda y las cuatro decisiones de cómputo.